International Journal of Forecasting 41 (2025) 1093–1111

Contents lists available at ScienceDirect

International Journal of Forecasting

journal homepage: www.elsevier.com/locate/ijforecast

SpotV2Net: Multivariate intraday spot volatility forecasting via vol-of-vol-informed graph attention networks

Alessio Brini [a,*], Giacomo Toscano [b]

[a] Duke University Pratt School of Engineering, 305 Teer Engineering Building Box 90271, Durham, NC 27708, USA
[b] Department of Economics and Management, University of Florence, Via delle Pandette 32, Firenze, 50127, Italy

ARTICLE INFO

Article history:
Dataset link: https://github.com/Alessiobrini/SpotV2Net

Keywords:
Multivariate spot volatility forecasting
Graph neural networks
Graph attention networks
Spot volatility
Spot volatility of volatility
Non-parametric Fourier estimators

ABSTRACT

This paper introduces SpotV2Net, a multivariate intraday spot volatility forecasting model based on a graph attention network architecture. SpotV2Net represents assets as nodes within a graph and includes non-parametric high-frequency Fourier estimates of the spot volatility and co-volatility as node features. Further, it incorporates Fourier estimates of the spot volatility of volatility and co-volatility of volatility as features for node edges, to capture spillover effects. We test the forecasting accuracy of SpotV2Net in an extensive empirical exercise, conducted with the components of the Dow Jones Industrial Average index. The results we obtain suggest that SpotV2Net yields statistically significant gains in forecasting accuracy, for both single-step and multi-step forecasts, compared to a panel heterogeneous autoregressive model and alternative machine-learning models. To interpret the forecasts produced by SpotV2Net, we employ GNNExplainer (Ying et al., 2019), a model-agnostic interpretability tool, and thereby uncover subgraphs that are critical to a node's predictions.

© 2024 International Institute of Forecasters. Published by Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

1. Introduction

Volatility forecasting has long been at the core of the literature on financial econometrics, as scholars and practitioners have been continually seeking accurate and robust asset volatility forecasts for applications in different areas, including option pricing, portfolio optimization, and financial risk management (see, respectively, Christoffersen and Diebold (2000), Bandi, Russell, and Yang (2008), and Becker, Clements, Doolan, and Hurn (2015), among many others).

Even though the focus of academic research has mainly been on forecasting one-day-ahead volatilities (see, for instance, Andersen, Bollerslev, Christoffersen, and Diebold (2006), Brailsford and Faff (1996), Poon and Granger (2003), Satchell and Knight (2011)), in recent years, the fast development of the high-frequency trading industry has emphasized the need for reliable intraday volatility forecasts, see Engle and Sokalska (2012), Rossi and Fantazzini (2015), Zhang, Zhang, Cucuringu, and Qian (2023b). The availability of accurate intraday volatility forecasts can be essential for the decision-making process of market operators in different circumstances.

Firstly, short-term volatility forecasts may be exploited to perform timely risk management tasks within the daily horizon (see, e.g., Madhusudan and Samit (2019), Rice, Wirjanto, and Zhao (2020)). Indeed, intraday volatility prediction can anticipate sudden market shifts, especially after significant events, and provide valuable information for traders and portfolio managers who rely on real-time risk assessments to adjust positions or hedge strategies promptly. Secondly, with the growth of algorithmic and high-frequency trading, modeling and forecasting intraday volatility dynamics has become foundational to guide profit-making strategies that hinge on short-term price movements (see, e.g., Behrendt and Schmidt (2018), Goldstein, Kwan, and Philip (2023), Liu, Pantelous, and von Mettenheim (2018), Mariotti, Lillo, and Toscano (2023)).

* Corresponding author.
E-mail addresses: alessio.brini@duke.edu (A. Brini), giacomo.toscano@unifi.it (G. Toscano).

https://doi.org/10.1016/j.ijforecast.2024.11.004
0169-2070/© 2024 International Institute of Forecasters. Published by Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

For instance, anticipating short-term volatility changes could allow high-frequency traders to improve optimal execution, exploit sudden shifts in market sentiment, and optimally plan market entrance/exit. At the same time, financial institutions may exploit accurate short-term volatility predictions to adjust trading margins in a timely way. Similarly, market makers could use intraday volatility forecasts to optimally plan their operations in order to control bid–ask spreads and guarantee adequate liquidity.

Beyond the trading setting, anticipating intraday volatility changes may be instrumental for the early detection of anomalies at the microstructural level (see, for example, Ligot, Gillet, and Veryzhenko (2021), Xue and Gençay (2012)). Moreover, regulators could use short-term volatility forecasts to anticipate the outset of instability conditions (Allaj & Sanfelici, 2023).

In this paper, we introduce a novel approach to multivariate intraday volatility forecasting based on graph neural networks (GNNs). GNNs represent a specific instance of neural network architecture that can act on data structured as graphs. They have been successfully employed to model several complex multidimensional dynamic problems in different fields, such as traffic flows (Diao, Wang, Zhang, Liu, Xie, & He, 2019; Jiang & Luo, 2022; Li & Zhu, 2021), recommendation systems (Huang et al., 2021; Xia, Huang, Xu, Dai, & Bo, 2022; Ying, He, Chen, Eksombatchai, Hamilton, & Leskovec, 2018), social networks (Fan et al., 2019; Minet al., 2021), supply chains (Gopal & Chang, 2021), international trade (Monken, Haberkorn, Gopinath, Freeman, & Batarseh, 2021; Panford-Quainoo, Bose, & Defferrard, 2020), interbank markets (Liu, Cheng, & Jiang, 2023), enterprise bankruptcy (Zhao et al., 2022), and tax evasion (Shiet al., 2023). The dynamics of asset volatilities in a high-dimensional intraday setting may be seen as a complex system, based on the empirical evidence that suggests the presence of intraday co-movements and spillovers effects across different sectors and markets, see, e.g., Fassas and Siriopoulos (2019), Golosnoy, Gribisch, and Liesenfeld (2015), Jawadi, Louhichi, and Idi. Cheffou (2015), Katsiampa, Corbet, and Lucey (2019), Naeem, Karim, Yarovaya, and Lucey (2023), Nishimura and Sun (2018). Hence, GNNs represent an effective modeling framework to capture the changing patterns in the intraday volatility of assets by leveraging data from various correlated securities.

In a multidimensional setting, we assume that each asset represents a node in a graph. The features (covariates) associated with each node comprise the instantaneous (or spot) volatility of the asset and the instantaneous co-volatilities with every other asset in the graph, reconstructed using the non-parametric Fourier methodology by Malliavin and Mancino (2002, 2009). Furthermore, to capture spillover effects, the edge features encapsulate information about the dynamic dependence between node pairs. Specifically, for any given edge, its features consist of Fourier estimates of the volatility of volatility of each of the assets connected by the edge and the co-volatility of their individual volatilities. In this regard, we note that different studies have shown that encompassing realized or implied volatility-of-volatility measures improves a model's performance in risk management tasks (see, e.g., Chen, Chordia, Chung, and Lin (2021), Huang, Schlag, Shaliastovich, and Thimme (2019), Li (2022) and forecasting applications (see, e.g., Campisi, Muzzioli, and Baets (2023), Catania and Proietti (2020), Ding (2023)). Node and edge features represent the inputs of the forecasting model. To account for (cross-) serial dependence, each input includes not only current values but also several lagged values. This is achieved by leveraging the capability of deep learning architectures to handle a large number of inputs (Jaegleet al., 2021; Menghani, 2023). In the specific case of GNNs, inputs can be placed in both the nodes and the edges (Wu et al., 2020a; Zhou et al., 2020). We refer to the resulting architecture that we obtain as a spot volatility and volatility-of-volatility network, abbreviated SpotV2Net.

The GNN architecture that we employ in this paper relies on an attention mechanism and thus falls within the graph attention network (GAT) framework proposed by Veličkovićet al. (2017). The attention mechanism allows for assigning different attention scores to different nodes in a graph, thereby enabling the forecasting model to attribute more weight to the information embedded into a specific subset of neighboring nodes. This adaptability can be crucial for the efficiency of empirical applications in which nodes are not likely to contribute equally to the forecast of the volatility of a given asset in a graph. Moreover, the GAT framework can inherently handle edge features because the attention mechanism also includes the latter in the calculation of attention scores. On the contrary, other types of GNN instances, such as graph convolutional networks (GCNs) (Kipf & Welling, 2016), spectral-based GNNs (Defferrard, Bresson, & Vandergheynst, 2016), and spatial-based GNNs (Hamilton, Ying, & Leskovec, 2017), operate with a fixed, predetermined weight for neighboring nodes and do not naturally handle edge features.

We test the SpotV2Net model in a universe of the 30 stocks that compose the Dow Jones Industrial Average (DJIA) index at the time of writing. Using one-second asset prices, we estimate model inputs, namely univariate and multivariate spot volatility and volatility-of-volatility time series, on a 30-minute grid. Then, we evaluate the performance of SpotV2Net's single-step and multi-step spot volatility forecasts and compare its accuracy with that of alternative models. These include a heterogeneous autoregressive (HAR) model for spot volatility,[1] and two machine learning (ML) models that do not handle graph structures, namely the extreme gradient boosting (XGB) model and the long short-term memory (LSTM) model. We find that the capability of SpotV2Net to learn from graph-like structured data and, in particular, to capture spillover effects via edge features yields statistically significant gains in forecasting accuracy with respect to the alternative models considered. Such gains are obtained with both single-step and multi-step forecasts. In this

[1] We adapt the HAR model by Corsi (2009) which is built to forecast daily integrated volatilities, to forecast intraday spot volatilities. The resulting model, which maintains a heterogeneous autoregressive structure, is termed HAR-Spot and is illustrated in Appendix A.

regard, we note that SpotV2Net is capable of producing multi-step forecasts without resorting to a recursive approach.

Furthermore, given the complexity of the GAT architecture employed, we address the issue of the interpretability of the predictions produced by the SpotV2Net model. Specifically, we employ the GNNExplainer framework (Ying, Bourgeois, You, Zitnik, & Leskovec, 2019) to identify relevant nodes and edges that influence the model's spot volatility predictions for a specific asset.

The paper is organized as follows. Section 2 reviews the relevant literature on multivariate volatility forecasting, while Section 3 introduces the notation and illustrates the assumptions behind the estimation of univariate and multivariate spot volatility and volatility-of-volatility time series. Section 4 motivates the use of a GAT framework with co-volatilities of volatilities as edge features for forecasting intraday multivariate spot volatilities. Section 5 then formally introduces the SpotV2Net model, while Section 6 recalls the definition of the Fourier spot estimators employed to reconstruct the inputs of SpotV2Net. Section 7 illustrates the empirical application of SpotV2Net to the universe of DJIA stocks, and Section 8 concludes. Finally, Appendix A briefly illustrates the modeling alternatives to SpotV2Net which are implemented for comparison in the empirical application of Section 7, while Appendix B provides information on setting hyperparameters.

2. Related literature

The complexity of financial markets explains the growing interest of the literature on multivariate models that can capture volatility co-movements among different securities. In this regard, see, for instance, Bauwens, Laurent, and Rombouts (2006), who provide a comprehensive survey of the multivariate extension of GARCH-like models–such as the diagonal VEC model (Bollerslev, Engle, & Wooldridge, 1988), the BEKK model (Engle & Kroner, 1995), the VAR-GARCH model (Ling & McAleer, 2003), and Wishart autoregression (WAR) (Gouriéroux, Jasiak, & Sufana, 2009)–and Wilms, Rombouts, and Croux (2021), who employ a multivariate version of the HAR model (Corsi, 2009) to capture volatility spillover effects among stock market indices.

Bollerslev, Meddahi, and Nyawa (2019), Caldeira, Moura, Nogales, and Santos (2017), Herskovic, Kelly, Lustig, and Va. Nieuwerburgh (2016, 2020) show the importance of these models for applications in asset pricing, portfolio optimization, and risk management. However, these models present a number of drawbacks. For example, the diagonal VEC, BEKK, and HAR models face significant computational challenges when scaling to a large number of assets. For instance, Callot, Kock, and Medeiros (2017) explored the curse of dimensionality in multivariate volatility models, noting that these models may lead to poor out-of-sample forecasts and high computational burdens, making the development of a more scalable solution a valuable approach in this area of study. Additionally, as pointed out by Zhang, Pu, Cucuringu, and Dong (2022a, 2023a), VAR-GARCH and WAR struggle to capture long-term dependencies in volatility time series.

Within a multivariate framework, forecasting an intraday spot measure of asset volatility is a critical area of exploration that has received little attention in the literature, to the best of our knowledge. Few works attempt to model intraday volatility through ARCH (Taylor & Xu, 1997) and GARCH (Engle & Sokalska, 2012) variants. However, their modeling focus is on integrated measures of volatilities, namely on realized volatilities (Andersen, Bollerslev, Diebold, & Ebens, 2001), rather than on spot estimates. Moreover, as pointed out by Christensen, Siggaard, and Veliyev (2023), traditional linear models may break down when the explanatory variables are strongly correlated, exhibit low signal-to-noise ratios, or if the underlying structure is nonlinear.

On this behalf, ML techniques, which have recently made an impact on the field of finance, represent an alternative approach to effectively handling these issues. The capability of ML models, especially deep neural networks, to handle high-dimensional data and approximate complex patterns has impacted key areas of finance, such as portfolio optimization (Heaton, Polson, & Witte, 2017; Lin & Taamouti, 2024; Ma, Han, & Wang, 2021; Zhang, Zohren, & Roberts, 2020), asset pricing (Chen, Pelger, & Zhu, 2023; Gu, Kelly, & Xiu, 2020; Wu, Brinton, Zhang, Pizzoferrato, Liu, & Cucuringu, 2021), and volatility forecasting (Bucci, 2020; Christensen et al., 2023; Liu, 2019; Xiong, Nichols, & Shen, 2015; Zhu, Bai, He, & Liu, 2023). For the specific problem of volatility forecasting, it is worth underlining that the findings by Christensen et al. (2023) suggest that several ML algorithms, despite being implemented with minimal hyperparameter tuning, yield improved forecasting performance compared to models in the HAR lineage, especially at longer horizons. The authors attribute such forecast accuracy gains to higher persistence in the ML models –which helps to approximate the long memory of realized variance– and the capability of ML models to encompass the contribution from additional predictors, besides past volatility lags.

GNNs are part of the ML-based approaches that can play a significant role in enhancing financial applications, see, e.g., the review by Wang, Zhang, Xiao, and Song (2021) and Zhang, Chen, Miao, and Liu (2022b). Recent studies (see Chen and Robert (2022), Djanga, Cucuringu, and Zhang (2023), Reisenhofer, Bayer, and Hautsch (2022), Zhang et al. (2022a)) employ GNN-based models to forecast integrated measures of asset volatility, illustrating the potential of graph-based methods to capture complex inter-asset relationships, including non-linear volatility spillover effects (Zhang et al., 2023a). Similarly, Wu and Pan and Long et al. (2020b) and Cheng, Yang, Xiang, and Liu (2022) extend the scope of application for GNNs by employing the latter for forecasting financial time series. Building on these foundations, our research aims to extend the use of GNNs by including the attention mechanism typical of GATs. GAT architectures have already been employed to predict stock market movements (Cheng et al., 2022; Kimet al., 2019) and provide stock recommendations (Ying, Xu, Gao, Wang, & Li, 2020). However, few works have leveraged the flexibility of GATs for modeling spillover effects in financial markets. For instance, Cheng and Li (2021) model the momentum effect between firms

using an attribute-sensitive architecture similar to our approach, but they focus on solving a binary classification problem to predict the directional movements of stock prices, rather than forecasting asset volatilities.

3. Notation and assumptions

We consider a collection of N assets. For $i = 1, \dots, N$, we let $p_i(t)$ and $V_i(t)$ denote, respectively, the log-price and the spot volatility of the ith asset at time $t \in [0, T]$. We assume that the dynamics of the pair $(p_i(t), V_i(t))$ satisfy

$$\begin{cases} dp_i(t) = \mu_{i,1}(t)dt + \sqrt{V_i}(t)dW_{i,1}(t) \\ dV_i(t) = \mu_{i,2}(t)dt + \sqrt{\widetilde{V}_i}(t)dW_{i,2}(t) \end{cases},$$

where, for any $i \in \{1, \dots, N\}$ and for any $k \in \{1, 2\}$, $W_{i,k}$ is a Brownian motion on the filtered probability space $(\Omega, P, \mathcal{F}, (\mathcal{F}_t)_{t\in[0,T]})$, while the drift $\mu_{i,k}$ and the spot volatility of volatility $\widetilde{V}_i$ are continuous, bounded, and adapted stochastic processes defined on $(\Omega, P, \mathcal{F}, (\mathcal{F}_t)_{t\in[0,T]})$. The pairwise correlations between the Brownian motions $W_{1,1}, \dots, W_{N,1}, W_{1,2}, \dots, W_{N,2}$ are allowed to be time-dependent and random.

Let $\langle p_i, p_j\rangle_t$ denote the quadratic covariation between p_i and p_j on $[0, t]$. We define the spot co-volatility process as

$$C_{ij}(t) := \frac{d\langle p_i, p_j\rangle_t}{dt}$$

and note that $C_{ii}(t) = V_i(t)$. Similarly, we define the spot co-volatility-of-volatility process as

$$\widetilde{C}_{ij}(t) := \frac{d\langle V_i, V_j\rangle_t}{dt},$$

with $\widetilde{C}_{ii}(t) = \widetilde{V}_i(t)$.

The SpotV2Net model employs estimates of the discrete trajectories

$$\{C_{ij,t}\}_{t\in\mathcal{T}}, \quad \{V_{i,t}\}_{t\in\mathcal{T}}, \quad \{\widetilde{C}_{ij,t}\}_{t\in\mathcal{T}}, \quad \{\widetilde{V}_{i,t}\}_{t\in\mathcal{T}},$$
$$(i, j) \in \{1, \dots, N\} \times \{1, \dots, N\},\ i \neq j,$$

where $\mathcal{T} = \{0 = \tau_0 < \tau_1 < \cdots < T = \tau_B\}$, $B \in \mathbb{N}$. For ease of notation, we use, e.g., $V_{i,b}$ to indicate V_{i,τ_b}, $b = 0, 1, \dots, B$. Estimates are obtained using the Fourier methodology, which is detailed in Section 6. We denote the estimate of, e.g., $V_{i,b}$ by $\widehat{V}_{i,b}$.

4. Motivation

In the last two decades, different studies have highlighted the importance of including time-varying volatility-of-volatility effects in financial models to accurately capture price and volatility stylized facts and improve forecasting performance. For instance, in their seminal work, Bollerslev, Tauchen, and Zhou (2009) found that accounting for time-varying volatility-of-volatility effects is of primary importance for modeling and forecasting the temporal variation of expected returns. Furthermore, the findings by Corsi, Mittnik, Pigorsch, and Pigorsch (2008) suggest that allowing for time-varying volatility of volatility substantially improves both the fit and the predictive performance of traditional autoregressive models like the ARFIMA (Granger & Joyeux, 1980) and HAR models. More recently, other works have demonstrated the importance of the volatility of volatility in forecasting financial variables. Catania and Proietti (2020) proposed a score-driven model for the realized volatility and price dynamics with time-varying volatility-of-volatility effects and showed that the latter contributed to improving out-of-sample performance compared to a benchmark HAR-GARCH model. Additionally, Ding (2023) introduced a GARCH-type model that allows for conditional heteroskedasticity in the volatility of asset returns and showed that this model can capture returns stylized facts and yield more accurate volatility forecasts than commonly used models in the GARCH family. Moreover, beyond traditional autoregressive models, Campisi et al. (2023) found, by implementing several ML methods, that the implied volatility of volatility, as proxied by the CBOE VVIX index,[2] is a predictor, along with other market indices, of the future direction of the stock market.

The SpotV2Net model employs the information embedded in spot co-volatility-of-volatility paths to obtain accurate intraday spot volatility forecasts in a multivariate setting. Specifically, the model rests on the idea that instantaneous spillover effects between asset volatilities are naturally captured by the dynamics of the time-varying spot co-volatilities of volatility pairs.

Intuitively, SpotV2Net views asset spot volatilities at a given point in time as different nodes of a graph that are connected by edges embedding information about the contingent *strength* of the relationship between node pairs. Such information is naturally represented by the instantaneous dependence between two nodes (i.e., volatilities), as measured by the spot co-volatility between the nodes (i.e., the spot co-volatility of volatility). Given this structure, a time-varying co-volatility of volatility can be naturally seen as an indicator of the magnitude of instantaneous spillover effects.

We can illustrate the role of the co-volatility of volatility with an empirical example. Consider the two consequential financial events that occurred in March 2023, namely the collapses of Silicon Valley Bank (on March 10) and Signature Bank (on March 12). Given the pivotal role of banks in the financial system, these bank failures immediately initiated a volatility burst that spread from the US to international markets. However, not all stocks' volatilities were affected by spillover effects of the same magnitude.

For example, consider the following three US companies that are part of the Dow Jones Industrial Average index: American Express (AXP) and Visa (V), two companies that provide financial services, and Caterpillar (CAT), a manufacturing company. As illustrated in Fig. 1, all three companies experienced an increase in their spot volatility level right after the failures of Silicon Valley Bank and Signature Bank, showing the largest volatility peaks right after March 13. The largest volatility peaks were then followed by sequences of local peaks of lower magnitude and a progressive reversion to the volatility level observed at the beginning of the month.

[2] CBOEVVIXwebsite

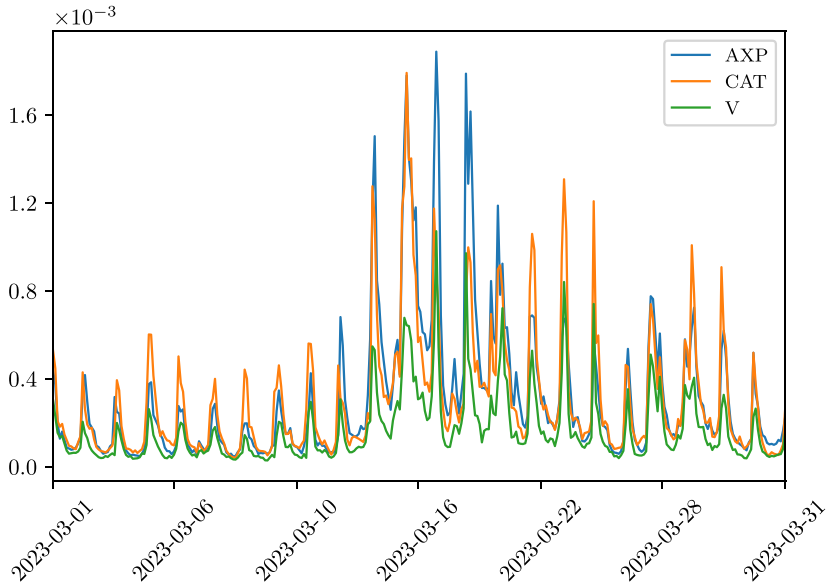

Fig. 1. Fourier spot volatility estimates $\{V_{i,t}\}_{t\in\mathcal{T}}$ for $i \in \{AXP, CAT, V\}$ on the 30-minute grid $\mathcal{T} = \{\tau_j = jT/13 : j = 0, 1, \ldots, 13\}$, where T is a 6.5-hour trading day. Estimates were obtained using one-second prices. Details about the Fourier estimator of the spot volatility are provided in Section 6.

However, by observing Fig. 2, one immediately notices that the pairwise co-volatilities of volatility behave very differently. In particular, the spot co-volatility of volatility between AXP and V shows two very large peaks near March 16 and March 17, while the co-volatility of volatility between each of these two stocks and CAT maintains substantially stable and lower values during March 2023. Such bursts in the spot co-volatility of volatility may indicate the presence of more pronounced instantaneous spillover effects between AXP and V, compared to the pairs AXP-CAT and V-CAT. This is not surprising, since AXP and V operate in the same sector, namely, financial services, which the collapses of Silicon Valley Bank and Signature Bank directly impacted. Instead, CAT is a well-diversified manufacturing company whose spot volatility may be less correlated with the volatility of companies operating in the banking and insurance industry. SpotV2Net can recognize such effects using the attention mechanism inherent in the GAT architecture. When predicting the future value of a specific node, the attention mechanism allows for assigning different weights to neighboring nodes, based not only on node features but also on features embedded in the connecting edges. Hence, the attention mechanism enables SpotV2Net to naturally capture the time-varying spillover effects between volatility pairs by including edge features. The next section describes the attention mechanism in detail.

5. The SpotV2Net model

GNNs are a family of deep learning models (Goodfellow, Bengio, & Courville, 2016) that can learn from data structured as graphs. They differ from other widely used neural network architectures such as convolutional neural networks (CNNs) (LeCun, Bengio, et al., 1995) and recurrent neural networks (RNNs) (Hochreiter & Schmidhuber, 1997; Rumelhart, Hinton, Williams, et al., 1985), which are designed to model for grid and sequence data, respectively.

The general idea of GNNs rests on the principle of iterative message passing, where graph nodes aggregate information from themselves and their connected nodes, referred to as neighbors. This aggregation can be repeated by stacking multiple layers, where each layer generates its own representation based on the output of the previous layer. These representations are then passed to a linear aggregation layer to perform node regression, i.e., obtaining output forecasts for each node. This end-to-end learning process allows for training the model to operate directly on graph-structured data.

The mathematical operation that produces the aggregation defines the type of GNN algorithm. Scarselli, Gori, Tsoi, Hagenbuchner, and Monfardini (2008) presented the first example of GNN modeling characterized by an iterative updating procedure for the node representation. Other models introduce improvements in computational efficiency, ease of training, and the ability to handle complex graph structures. For instance, graph convolutional networks (GCNs) allow the computation to be performed in parallel and follow a static layered structure similar to a feedforward neural network (Kipf & Welling, 2016). Furthermore, graph isomorphism networks (GINs) (Xu, Hu, Leskovec, & Jegelka, 2018) include fixed trainable weights that determine the different impacts of neighboring nodes when performing the prediction of a specific node.

The GAT (Veličkovićet al., 2017), on which SpotV2Net is based, introduces an attention mechanism that sets it apart from other models. While GCNs propagate and aggregate information uniformly from neighbors, and the GIN ensures that the network can effectively discern the

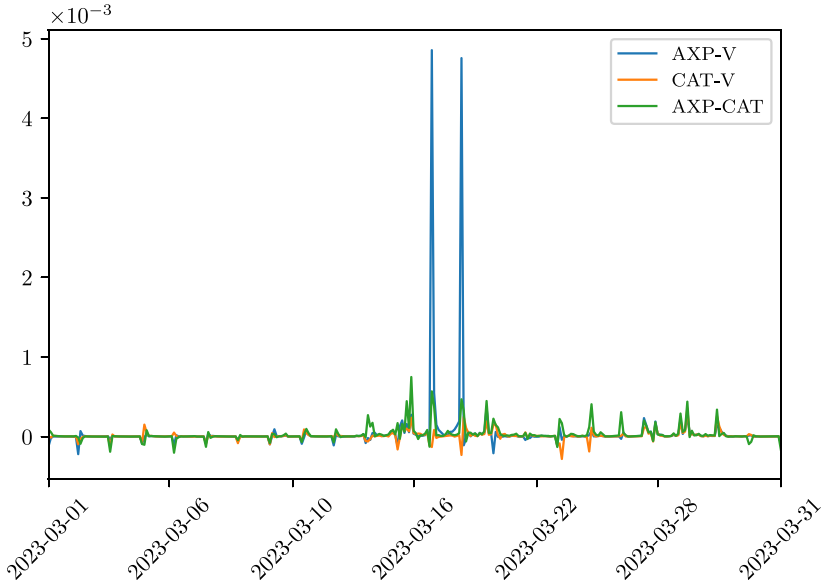

Fig. 2. Fourier spot co-volatility-of-volatility estimates $\{\widetilde{C}_{ij,t}\}_{t\in\mathcal{T}}$ for $i,j \in \{AXP, CAT, V\}$, $i \neq j$, on the 30-minute grid $\mathcal{T} = \{\tau_j = jT/13 : j = 0, 1, \ldots, 13\}$, where T is a 6.5-hour trading day. Estimates were obtained using one-second prices. Details about the Fourier estimator of the spot volatility are provided in Section 6.

relative significance of a node's inherent features as opposed to the aggregated features from neighboring nodes, the GAT empowers nodes to assign different weights to their neighbors based on their features and the features of the connecting edges. This dynamic mechanism builds on transformer-based natural language processing models (Vaswani et al., 2017).

Let us now consider the mechanism behind a GAT. Let $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ denote a graph structure, where $\mathcal{V} = \{\nu_1, \ldots, \nu_N\}$ is the set of the N nodes of the structure, and $\mathcal{E}$ is the set of edges, where $\varepsilon_{ij} = (\nu_i, \nu_j) \in \mathcal{E}$ denotes the edge connecting node ν_i and node ν_j. Further, let A denote the $N \times N$ binary adjacency matrix of the structure, where $A[i,j]$ is equal to 1 (respectively, 0) if ν_i and ν_j are connected (respectively, are not connected). The graph $\mathcal{G}$ is assumed to be fully connected. That is, all the entries of A are assumed to be equal to 1.

A GAT involves a sequence of layers, where the previous layer's input represents the next layer's output. The final layer is referred to as the prediction (output) layer. Intermediate layers are referred to as hidden layers. When implementing SpotV2Net, we optimize the number of hidden layers as a hyperparameter, determining that two hidden layers are optimal.

The input to the first GAT layer is the set of vectors representing node features, namely $x = \{x_1, x_2, \ldots, x_N\}$, $x_i \in \mathbb{R}^M$, where M indicates the number of features in each node. The GAT layer outputs a different representation of the initial set of node features, with different cardinality M', denoted by $x' = \{x'_1, x'_2, \ldots, x'_N\}$, $x'_i \in \mathbb{R}^{M'}$. We treat M' as a hyperparameter and optimize its value through the training phase.

Specifically, the operation performed by the GAT layer consists of the following steps. Firstly, the attention coefficient that indicates the importance of node j's features to node i, denoted by e_{ij}, is obtained as

$$e_{ij} = a(Wx_i, Wx_j),$$

where a is a $\mathbb{R}$-valued function defined on $\mathbb{R}^{M'} \times \mathbb{R}^{M'}$, and $W \in \mathbb{R}^{M' \times M}$ is a weight matrix. Then, attention coefficients are normalized to make the values comparable across nodes. The normalization is performed using the *softmax* function:

$$\alpha_{ij} = \mathrm{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k=1}^{N} \exp(e_{ik})}.$$

In our study, we follow the implementation by Veličkovićet al. (2017), so that a is a single-layer feedforward neural network with a LeakyReLU activation function (Maas, Hannun, Ng, et al., 2013) and weight vector $q \in \mathbb{R}^{2M'}$. The LeakyReLU function is defined as

$$\mathcal{L}(x) = \begin{cases} x & \text{if } x > 0 \\ cx & \text{if } x \leq 0 \end{cases},$$

where c is a positive constant. The resulting attention coefficient reads as

$$\alpha_{ij} = \frac{\exp(\mathcal{L}(q^T[Wx_i \parallel Wx_j]))}{\sum_{k=1}^{N} \exp(\mathcal{L}(q^T[Wx_i \parallel Wx_k]))},$$

where $\cdot^T$ represents transposition, and $\parallel$ indicates the concatenation operation.

Once the attention parameters are computed, the hidden representation for each node in the graph is expressed as

$$x'_i = \sigma\left(\sum_{j=1}^{N} \alpha_{ij} W x_j\right),$$

where σ represents a nonlinear activation function chosen as a hyperparameter of the model.

To improve the expressiveness[3] and accuracy of the training process (Vaswani et al., 2017), when implementing SpotV2Net we follow a multi-head attention mechanism that consists of a number K of independent attention mechanisms. Multiple heads perform the same operations on the same input but are initialized with different weights. This approach allows each head to learn distinct patterns from the inputs and capture various aspects of the data. The resulting hidden node representation is obtained via concatenation and reads as

$$x_i' = \|_{k=1}^{K} \sigma \left(\sum_{j=1}^{N} \alpha_{ij}^k W^k x_j \right),$$

where α_{ij}^k denotes normalized attention coefficients computed by the kth attention mechanism, and W^k is the corresponding weight matrix. To avoid an excessive increase in dimensionality, which could lead to overfitting (Veličkovićet al., 2017), concatenation is not performed if the hidden layer precedes the final prediction layer. Instead, in this case, an average is computed:

$$x_i' = \sigma \left(\frac{1}{K} \sum_{k=1}^{K} \sum_{j=1}^{N} \alpha_{ij}^k W^k x_j \right).$$

In the last layer, an affine transformation is applied to obtain the prediction. Specifically, the final layer yields the prediction for node i, denoted by $\hat{y}_i$, which reads as

$$\hat{y}_i = O x_i' + u,$$

where $O \in \mathbb{R}^{N \times M'}$ and $u \in \mathbb{R}^N$.

In the specific case of SpotV2Net, at a given time τ_b, we feed the GAT the node feature vectors

$$x_i = \left[\left\{ \widehat{V}_{i,b-l} \right\}_{l=0,\dots,L}, \left\{ \left\{ \widehat{C}_{ij,b-l} \right\}_{j=1,\dots,N; j \neq i} \right\}_{l=0,1,\dots,L} \right]. \quad (1)$$

That is, x_i includes contemporaneous values and all the lags up to L of the estimated volatility of the ith asset and the estimated co-volatility between the ith asset and all the other assets. Note that the number of lags L is not predetermined a priori, but rather treated as a hyperparameter subject to tuning (see the hyperparameter 'Number of Lags' in Appendix B). This allows for flexibility in dynamically capturing the long-memory property of the spot volatility.

Our application of the GAT layer also allows for edge features, i.e., information regarding the dependence between two nodes in a graph. Specifically, we have a set of edge features $x^e = \{x_{ij}^e \mid i < j\}$, $\quad x_{ij}^e \in \mathbb{R}^E$. We set

$$x_{ij}^e = \left[\left\{ \widehat{\widehat{V}}_{i,b-l} \right\}_{l=0,\dots,L}, \left\{ \widehat{\widehat{V}}_{j,b-l} \right\}_{l=0,\dots,L}, \left\{ \widehat{\widehat{C}}_{ij,b-l} \right\}_{l=0,1,\dots,L} \right]. \quad (2)$$

That is, x_{ij}^e includes contemporaneous values and all the lags up to L of the estimated volatility of volatility of the ith asset and the jth asset, along with the estimated co-volatility of volatility between the ith asset and the jth asset. Fig. 3 illustrates the general structure of SpotV2Net's input graph in the three-dimensional case.[4]

To include edge features in a GAT, it is sufficient to modify the attention mechanism so that the function $a : \mathbb{R}^{M'} \times \mathbb{R}^{M'} \to \mathbb{R}$ becomes $a' : \mathbb{R}^{M'} \times \mathbb{R}^{M'} \times \mathbb{R}^{E'} \to \mathbb{R}$. The resulting normalized attention coefficient then reads as

$$\alpha_{ij}' = \frac{\exp\left(\mathcal{L}\left(q'^T [W x_i \parallel W x_j \parallel U x_{ij}^e]\right)\right)}{\sum_{k=1}^{N} \exp\left(\mathcal{L}\left(q'^T [W x_i \parallel W x_k \parallel U x_{ik}^e]\right)\right)},$$

where $U \in \mathbb{R}^{E' \times E}$ is the weight matrix for the linear transformation of the edge features, and E' is the dimension of the transformed edge features.

6. Fourier estimators

In this section, we briefly illustrate the Fourier estimation method, which allows us to efficiently reconstruct the entries of node and edge feature vectors defined, respectively, in (1) and (2). The Fourier non-parametric estimation method, originally introduced by Malliavin and Mancino (2002, 2009), allows for reconstructing the latent spot co-volatility paths from high-frequency price observations. The method is particularly apt for estimating spot co-volatilities in the presence of microstructure noise, as it removes the influence of the latter by simply cutting off the highest coefficients of the price increment from the convolution that yields the co-volatility coefficients (see Eq. (3)), without the need for bias correction.

The Fourier spot volatility estimator has been shown to achieve the optimal rate of convergence in the presence of noise, see Mancino, Mariotti, and Toscano (2022).[5] Its finite-sample efficiency at high frequencies is supported by the studies of Mancino et al. (2022), Mancino and Recchioni (2015), Mariotti et al. (2023).

The Fourier method also allows us to estimate the co-volatility-of-volatility path (Barucci & Mancino, 2010). It is worth noting that the Fourier spot co-volatility-of-volatility estimator does not require the pre-estimation of the spot volatility path but only the computation of integrated quantities, namely the coefficients of the co-volatility. This feature enhances the finite-sample efficiency of the Fourier methodology, compared to alternative approaches that require the pre-estimation of the spot volatility path, see Toscano, Livieri, Mancino, and Marmi (2022). The consistency of the Fourier spot volatility-of-volatility estimator with noisy prices is proven in Mancino et al. (2022). Its finite-sample performance at high frequencies is studied in Toscano (2022).

[3] Expressiveness (or capacity) refers to a model's ability to capture complex patterns and relationships in the data. For more details, see Chapter 5 of Goodfellow et al. (2016).

[4] This simplified, low-dimensional representation is intended to illustrate the process clearly, avoiding the cluttered figure that would result from using higher dimensions. However, the inherent scalability of the GNN architecture allows SpotV2Net to be applied effortlessly to a much larger cross-section of assets. Accordingly, in the empirical application presented in Section 7, we employ SpotV2Net to model the spot volatility of all of the 30 constituents of the DJIA index.

[5] This result is trivially extended to the Fourier spot co-volatility estimator if prices are sampled on a synchronous grid, as is the case in the empirical application of Section 7.

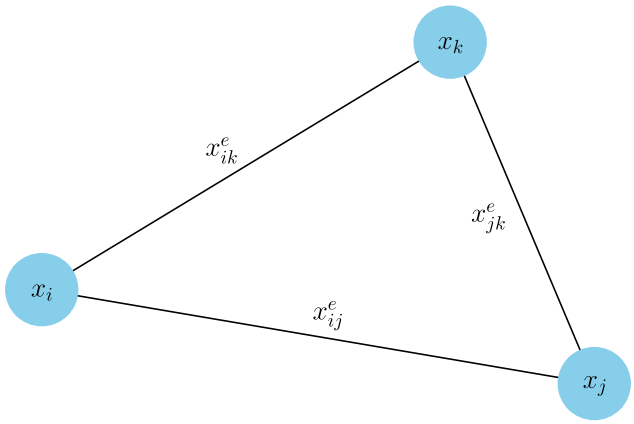

Fig. 3. Three-dimensional example of the fully connected graph topology of SpotV2Net. The three nodes are denoted by i, j, and k. The vectors x_* (where $* = i, j, k$) and x^e_{**} (where $** = ij, ik, jk$) are defined in Eqs. (1) and (2), respectively.

6.1. Fourier estimator of the volatility and co-volatility

Without loss of generality, consider the case with just two assets, that is $N = 2$. For any $i = 1, 2$, assume that the log-price process p_i is observable on the grid $\mathcal{D}_n := \{0 = t_{0,n} < t_{1,n} \ldots < t_{n-1,n} < T = t_{n,n}\}$ such that $max_{s\in\{1,\ldots,n\}}|t_{s,n} - t_{s-1,n}| \to 0$ as $n \to \infty$.[6]

For $|k| \le N_c$, an estimator of the kth Fourier coefficient of the co-volatility is given by the convolution

$$c_k\left(C_{n,N_c}\right) = \frac{T}{2N_c+1}\sum_{|l|\le N_c} c_l\left(dp_{1,n}\right)c_{k-l}\left(dp_{2,n}\right), \tag{3}$$

where, for any integer k such that $|k| \le 2N_c$, $c_k\left(dp_{i,n}\right)$ is the kth discrete Fourier coefficient of the log-return of the ith asset, namely

$$c_k\left(dp_{i,n}\right) := \frac{1}{T}\sum_{l=0}^{n-1} e^{-\mathrm{i}k\frac{2\pi}{T}t_{l,n}}\left(p_i(t_{l+1,n}) - p_i(t_{l,n})\right), \quad i = 1, 2.$$

Similarly, for $|k| \le N_{v_i}$, an estimator of the kth Fourier coefficient of the volatility of the ith asset is given by

$$c_k\left(V_{n,N_{v_i}}\right) = \frac{T}{2N_{v_i}+1}\sum_{|l|\le N_{v_i}} c_l\left(dp_{i,n}\right)c_{k-l}\left(dp_{i,n}\right), \quad i = 1, 2.$$

Once the Fourier coefficients of the co-volatility and the individual volatilities have been computed, applying the Fourier–Fejér inversion formula yields estimators of the co-volatility and volatility paths. Specifically, the Fourier estimators of the spot co-volatility and volatility at time $\tau_b \in \mathcal{T}$ are defined as

$$\widehat{C}_{12,b} = \sum_{|k|<M_c}\left(1 - \frac{|k|}{M_c}\right)c_k\left(C_{n,N_c}\right)e^{\mathrm{i}k\frac{2\pi}{T}\tau_b},$$

and

$$\widehat{V}_{i,b} = \sum_{|k|<M_{v_i}}\left(1 - \frac{|k|}{M_{v_i}}\right)c_k\left(V_{n,N_{v_i}}\right)e^{\mathrm{i}k\frac{2\pi}{T}\tau_b}, \quad i = 1, 2,$$

where $M_c < N_c$ and $M_{v_i} < N_{v_i}$.

6.2. Fourier estimator of the volatility of volatility and the co-volatility of volatility

The knowledge of the Fourier coefficients of the latent spot co-volatility and volatility allows for treating the latter as observable processes and iterating the procedure for computing the Fourier coefficients of the co-volatility of volatility and the volatility of volatility. Estimators of the kth Fourier coefficient of the co-volatility of volatility and the kth Fourier coefficient of the volatility of volatility are respectively defined as

$$c_k\left(\widetilde{C}_{n,N_c,S_c}\right) = \frac{T}{2S_c+1}\sum_{|l|\le S_c} l(l-k)c_l\left(C_{n,N_c}\right)c_{k-l}\left(C_{n,N_c}\right),$$

and

$$c_k\left(\widetilde{V}_{n,N_{v_i},S_{v_i}}\right) = \frac{T}{2S_{v_i}+1}\sum_{|l|\le S_{v_i}} l(l-k)c_l\left(V_{n,N_{v_i}}\right)c_{k-l}\left(V_{n,N_{v_i}}\right),$$

$$i = 1, 2,$$

where $S_c < N_c$ and $S_{v_i} < N_{v_i}$. The Fourier estimators of the spot co-volatility of volatility and volatility of volatility at time $\tau_b \in \mathcal{T}$ are then defined as

$$\widehat{\widetilde{C}}_{12,b} = \sum_{|k|<L_c}\left(1 - \frac{|k|}{L_c}\right)c_k\left(\widetilde{C}_{n,N_c,S_c}\right)e^{\mathrm{i}k\frac{2\pi}{T}\tau_b},$$

and

$$\widehat{\widetilde{V}}_{i,b} = \sum_{|k|<L_{v_i}}\left(1 - \frac{|k|}{L_{v_i}}\right)c_k\left(\widetilde{V}_{n,N_{v_i},S_{v_i}}\right)e^{\mathrm{i}k\frac{2\pi}{T}\tau_b}, \quad i = 1, 2,$$

where $L_c < S_c$ and $L_{v_i} < S_{v_i}$.

7. Empirical study: Application to the universe of DJIA stocks

In this section, we employ SpotV2Net to predict the intraday spot volatilities of the 30 components of the Dow Jones Industrial Average (DJIA) index. After briefly

[6] For simplicity of the exposition, we assume that two price series are synchronous, that is, are observed on the same grid $\mathcal{D}_n$, containing $n+1$ points.

describing the dataset and the estimation procedure for obtaining univariate and multivariate spot volatility and volatility-of-volatility time series from high-frequency prices, we evaluate the accuracy of the SpotV2Net model's single-step and multi-step forecasts. In the case of single-step forecasts, we employ GNNExplainer (Ying et al., 2019) to obtain insight into the most influential subgraphs related to a specific node's prediction.

7.1. Data preparation and estimation of univariate and multivariate spot volatility and the volatility-of-volatility series

We collected the raw price data for the 30 components of the DJIA index from the TAQ – Millisecond Consolidated Trades database, which we accessed through Wharton Research Data Services (WRDS). The dataset covers the period from June 1st, 2020, to May 10th, 2023, corresponding to 737 days.

For the implementation of the Fourier estimator, we proceed as follows. Starting from tick data, we filter the transactions to include only those executed on the New York Stock Exchange (NYSE) and exclude transactions that occurred outside the operating market hours from 9:30 a.m. to 4:00 p.m. Then, we resample the tick data to the one-second frequency, selecting the price associated with the transaction closest to each second's onset. The resulting sample size employed in the implementation of the estimators is $n = 23{,}400$ per day. Note that we measure time in days and set T equal to 1. The cutting frequencies N_{v_i}, M_{v_i}, S_{v_i}, and L_{v_i} (for the ith asset) and N_c, M_c, S_c, and L_c (for each pair of assets) are selected using guidance from the paper by Sanfelici and Toscano (2024), who introduced a MATLAB library for the implementation of Fourier estimators.[7] Univariate and multivariate spot volatility and volatility-of-volatility estimates are obtained on the 30-minute grid.

As the output of the univariate estimation process, for each of the $N = 30$ DJIA components, we obtain a time series that contains 10,318 spot volatility values and a time series with the same length that contains spot volatility-of-volatility values.[8] Furthermore, the output of the multivariate estimation process includes $\frac{1}{2}N \times (N-1)$ time series with length 10,318 for the spot co-volatilities, and other $\frac{1}{2}N \times (N-1)$ time series of the same length for the spot co-volatilities of volatility.

Fourier estimates time series are partitioned into three sets: training, validation, and test. Table 1 outlines the start and end dates of each set, along with the proportion of observations included in each set. For each set, every 30 min we set up a static fully connected graph which includes contemporaneous and lagged Fourier estimates as node and edge features, according to the structure outlined in Section 5.

Table 1
The table delineates the periods of the dataset allocated for the training, validation, and test sets, with the corresponding start and end dates. It also details the number of 30-minute observations employed in each period and their proportions within the dataset.

	Training	Validation	Test
Start	06/01/2020	07/21/2022	10/15/2022
End	07/20/2022	10/14/2022	05/10/2023
# of 30-min obs.	7518	840	1960
Proportion	73%	8%	19%

7.2. Single-step forecast

In this section, we present the results of the application of SpotV2Net for predicting the 30-minute-ahead spot volatility of the entire cross-section of DJIA constituents. We train the model using the data-splitting strategy detailed in Table 1 and use the validation set to fine-tune the hyperparameter choices. The test set is kept completely separate and is used for out-of-sample forecast evaluation. The selection of hyperparameters is documented in Appendix B.

Note that our focus is on predicting volatilities, i.e., the diagonal elements of the covariance matrix, rather than the entire covariance matrix. While the model architecture is inherently flexible and could be adapted to forecast the entire covariance matrix, we leave this extension to future research. This decision was motivated by the challenge posed by the curse of dimensionality, which causes the number of parameters in the model to increase rapidly with the cross-sectional dimension, with negative consequences regarding computational costs.

We compare the forecasting accuracy of SpotV2Net with that of three alternative models, detailed in Appendix A: the panel HAR-Spot model, the XGB model, and the LSTM model. We also compare the forecasting accuracy of SpotV2Net with that of a modified version of the latter that does not accept edge features as input but only node features. We refer to it as SpotV2Net with no edge features, shortened to SpotV2Net-NE. Table 2 shows aggregate values of the mean squared error (MSE):

$$\frac{1}{T_{\text{test}}} \sum_{j=1}^{T_{\text{test}}} \frac{1}{N} \sum_{i=1}^{N} \left(\widehat{V}_{i,j} - \widehat{V}^*_{i,j}\right)^2, \tag{4}$$

and the QLIKE:

$$\frac{1}{T_{\text{test}}} \sum_{j=1}^{T_{\text{test}}} \frac{1}{N} \sum_{i=1}^{N} \left[\frac{\widehat{V}_{i,j}}{\widehat{V}^*_{i,j}} - \log \frac{\widehat{V}_{i,j}}{\widehat{V}^*_{i,j}} - 1\right], \tag{5}$$

where $\widehat{V}^*_{i,j}$ denotes the forecast at time τ_{j-1} of the spot volatility of ith asset at time τ_j, and T_{test} is the length of the test period as indicated in Table 1. When the validation period is considered, T_{test} is replaced by T_{val}, which indicates the length of the validation period.

Aggregate performance measures in Table 2 suggest that SpotV2Net outperforms the HAR-Spot model and

[7] The Fourier method is not consistent in the presence of jumps. To remove the effect of jumps from the estimation, we identify one-second returns whose absolute value is larger than a given threshold ϑ_n and replace them with zeros. Letting $\vartheta_n = \beta\,(T/n)^{\alpha}$, the constants that determine the threshold are selected using numerical simulations. As a result, we find that suitable choices are $\beta = 0.5$ and $\alpha = 0.5$.

[8] The dataset covers 737 days. For each day (i.e., 6.5 h) we obtain 14 estimates on the 30-minute grid (note that the last estimate was performed at 3:59 p.m., instead of 4:00 p.m., to reduce the effects of periodicity on Fourier estimates). Hence the total of $737 \times 14 = 10{,}318$ estimates.

Table 2
Aggregate values of MSE and QLIKE in Eqs. (4) and (5) for the SpotV2Net model and the alternative models during the validation and test phases. An asterisk (*) indicates models that are included in the MCS at the 5% significance level.

	Validation		Test	
Single-step model	MSE	QLIKE	MSE	QLIKE
HAR-Spot	3.797e−08	0.263	5.089e−08	0.343
XGB	3.148e−08	0.217	6.232e−08	0.429
LSTM	2.629e−08	0.184	5.198e−08	0.359
SpotV2Net-NE	2.513e−08	0.173	5.115e−08	0.354
SpotV2Net	2.079e−08*	0.137*	4.273e−08*	0.286*

the other ML models considered, namely XGB and LSTM, during both the validation and test periods. Further, while the LSTM and XGB models represent an improvement in forecasting accuracy compared to the HAR-Spot model in the validation period, the latter shows better performance, compared to LSTM and XGB, in the test period. SpotV2Net-NE is less accurate than SpotV2Net in both the validation and the test phases. Moreover, it is worth noting that SpotV2Net-NE still outperforms HAR-Spot and the other ML methods in the validation period. Instead, in the test period, SpotV2Net-NE performs worse than HAR-Spot, but still better than LSTM and XGB. These findings underline the crucial role of edge features (that is, the values of the spot co-volatility-of-volatility matrix) in capturing spillover effects and improving forecasting accuracy. Note that the improved performance by all models in the validation period is due to the fact that the validation employs data for fine-tuning model hyperparameters.

Table 2 also reports the results of the model confidence set (MCS) (Hansen, Lunde, & Nason, 2011), which is applied column-wise to identify a subset of models that demonstrate superior performance with a 95% level of confidence. We implemented the MCS with 5000 bootstrap replications, using the range statistic to compute the p-values. The results show that the subset of superior models contains only SpotV2Net, indicating that the latter yields superior predictive performance compared to the other models considered, with a level of confidence of 95%.

Moreover, we employ the Diebold–Mariano test (Diebold & Mariano, 2002) to compare the forecasting accuracy between models at the 5% significance level. The test statistics are provided in Tables 3 and 4 for the validation and test sets, respectively. The results of the Diebold–Mariano test suggest that the gain in forecasting accuracy provided by SpotV2Net is statistically significant.

7.3. Model interpretation

The complexity of the GAT architecture, on which SpotV2Net is based, suggests using specific methodologies to enhance interpretability. Indeed, model interpretation can be particularly important for applications in the financial domain. In particular, it may be relevant to pinpoint which nodes (i.e., which asset volatilities) are the most influential in predicting the volatility of a specific node, thereby obtaining insight into spillover effects and volatility transmission channels captured by SpotV2Net.

Table 3
Comparison of single-step forecasting accuracy using the Diebold–Mariano test with MSE (top) and QLIKE (bottom) losses in the validation period. Positive numbers indicate that the column model outperforms the row model. An asterisk (*) denotes a significance level of 5%.

Validation period (MSE)				
	XGB	LSTM	SpotV2Net-NE	SpotV2Net
HAR-Spot	6.32*	40.23*	42.12*	45.18*
XGB	–	5.50*	6.16*	25.94*
LSTM	–	–	2.34*	9.08*
SpotV2Net-NE	–	–	–	6.12*

Validation period (QLIKE)				
	XGB	LSTM	SpotV2Net-NE	SpotV2Net
HAR-Spot	13.11*	21.05*	23.45*	28.14*
XGB	–	12.32*	13.06*	20.06*
LSTM	–	–	5.67*	12.43*
SpotV2Net-NE	–	–	–	8.91*

Table 4
Comparison of single-step forecasting accuracy using the Diebold–Mariano test with MSE (top) and QLIKE (bottom) losses in the test period. Positive numbers indicate that the column model outperforms the row model. An asterisk (*) denotes a significance level of 5%.

Test period (MSE)				
	XGB	LSTM	SpotV2Net-NE	SpotV2Net
HAR-Spot	−42.69*	−1.11	−0.38	6.71*
XGB	–	37.89*	43.25*	52.12*
LSTM	–	–	0.71	11.87*
SpotV2Net-NE	–	–	–	7.23*

Test period (QLIKE)				
	XGB	LSTM	SpotV2Net-NE	SpotV2Net
HAR-Spot	−44.34*	−3.17*	−2.41*	6.57*
XGB	–	22.98*	26.15*	32.54*
LSTM	–	–	0.55	10.62*
SpotV2Net-NE	–	–	–	7.05*

For this purpose, we employ GNNExplainer (Ying et al., 2019), a model-agnostic approach that provides interpretable explanations for the predictions of GNN-based models. Specifically, GNNExplainer can identify the subgraph[9] deemed the most influential for the model's prediction of a specific node.

We briefly illustrate how GNNExplainer works. Consider a trained GNN-based model, denoted by $\mathcal{S}$, and let $\mathcal{G}$ denote the input graph of $\mathcal{S}$, composed of N nodes. Further, let $\hat{y}_{i,\mathcal{S}(\mathcal{G})}$ denote the prediction produced by $\mathcal{S}$ for the target variable y_i associated with the ith node of $\mathcal{G}$. For brevity, we use the shorthand notation $\hat{y}_i := \hat{y}_{i,\mathcal{S}(\mathcal{G})}$. In the case of SpotV2Net, $\hat{y}_i$ and y_i will be, respectively, the spot volatility prediction and the actual spot volatility value associated with the ith node of the input graph described in Section 5.

For a given node i of $\mathcal{G}$, GNNExplainer aims to select the subgraph $\mathcal{G}^*$, composed of $N^* < N$ nodes, that maximizes the mutual information (MI) between the prediction $\hat{y}_i$ and the prediction $\hat{y}_i^* := \hat{y}_{i,\mathcal{S}(\mathcal{G}^*)}$, which indicates the

[9] A subgraph is defined as a graph consisting of a subset of the nodes and edges of the original graph.

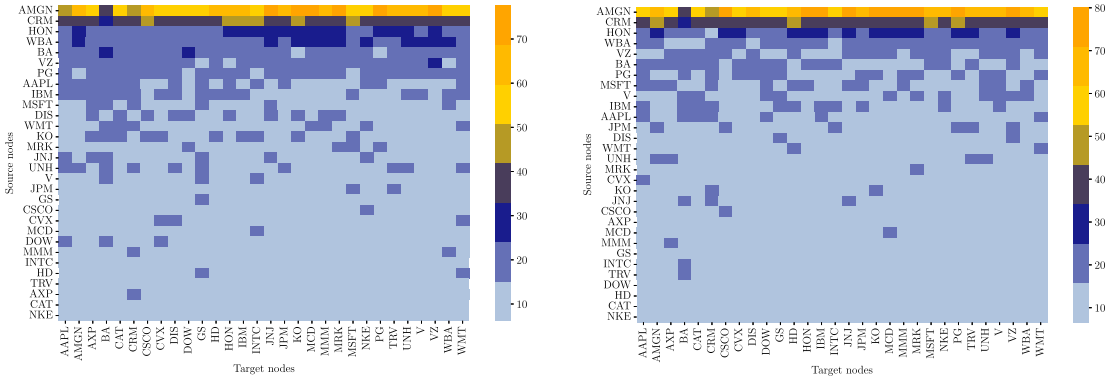

Fig. 4. Heatmaps assigning to nodes on the y-axis ('source nodes') the frequency (in percentage terms) by which GNNExplainer includes them in the most influential subgraphs (with $N^* = 5$ nodes) for predicting the spot volatility of nodes on the x-axis ('target nodes'), in the validation set (left panel) and test set (right panel). For each target node, GNNExplainer is implemented at every timestamp (i.e., single-step forecasts). The frequency of inclusion is computed by counting the number of timestamps (i.e., single-step forecast) for which a node is retained in the subgraph of the most influential nodes.

prediction produced by $\mathcal{S}$ for node i when its input is restricted to the subgraph $\mathcal{G}^*$.

Let $p_i := p_{i,\mathcal{S}(\mathcal{G})}$ and $p_i^* := p_{i,\mathcal{S}(\mathcal{G}^*)}^*$ denote, respectively, the density of the forecasts $\hat{y}_i$ and $\hat{y}_i^*$. The corresponding supports are denoted by $\mathcal{Y} \subseteq \mathbb{R}$ and $\mathcal{Y}^* \subseteq \mathbb{R}$. The optimization problem solved by GNNExplainer reads as

$$\max_{\mathcal{G}^*} MI_{i,\mathcal{G},\mathcal{G}^*} = h(\hat{y}_i) - h(\hat{y}_i^*), \tag{6}$$

where $h(\hat{y}_i) = -\int_{\mathcal{Y}} p_i(x) \log p_{i(x)} dx$ and $h(\hat{y}_i^*) = -\int_{\mathcal{Y}^*} p_i^*(x) \log p_i^*(x) dx$ denote, respectively, the entropy of $\hat{y}_i$ and $\hat{y}_i^*$. As the entropy of a forecast represents the level of uncertainty related to the forecast's outcomes, $MI_{i,\mathcal{G},\mathcal{G}^*}$ in Eq. (6) quantifies the difference in such uncertainty between the case when the input graph of $\mathcal{S}$ is the full graph $\mathcal{G}$ and the case when the input graph of $\mathcal{S}$ is restricted to $\mathcal{G}^*$. The larger the difference in Eq. (6), the more the subgraph $\mathcal{G}^*$ is deemed influential in predicting the target y_i. In other words, if removing the uncertainty ascribed to subgraph $\mathcal{G}^*$ maximizes the uncertainty related to the forecast $\hat{y}_i$, then the subgraph $\mathcal{G}^*$ is deemed a relevant *counterfactual* explanation of the forecast $\hat{y}_i$ (see Section 4.1 of Ying et al. (2019) for further details).

Since $h(\hat{y}_i)$ is independent of $\mathcal{G}^*$, the maximization problem in Eq. (6) is equivalent to the minimization problem $\min_{\mathcal{G}^*} h(\hat{y}_i^*)$.

For the interpretation of the single-step forecasts produced by SpotV2Net for the spot volatilities of the DIJA constituents, we implement GNNExplainer for each of the $N = 30$ nodes (i.e., asset spot volatilities) of the input graph, with the aim of extrapolating, for each node, a subgraph composed of the $N^* = 5$ nodes representing the most influential nodes for predicting the node's spot volatility. Note that we use GNNExplainer to identify the most influential subgraph, not to isolate the most influential individual features of specific nodes. This approach provides a global perspective on the relevance of a particular subgraph based on the entire information set embedded in its nodes and edges. This way, GNNExplainer complements traditional econometric methods, which typically emphasize the statistical significance of the contribution of individual features. The implementation is repeated at each time step.[10]

The results of the implementation of GNNExplainer are summarized in Fig. 4 using two heatmaps, one for the validation set and one for the test set, which illustrate the frequency (in percentage terms) of the inclusion of nodes into the subgraphs of the most influential nodes for predicting the spot volatility of a given asset.

Fig. 4 shows that Amgen (AMGN), Salesforce (CRM), and Honeywell (HON) are deemed the three most influential nodes for predicting the intraday spot volatility of all stocks in the dataset, in both the validation and test phases. Interestingly, these three companies became part of the DJIA index at the end of August 2020, near the beginning of the dataset, which starts in June 2020. More specifically, GNNExplainer suggests that the univariate and multivariate volatility and volatility of volatility of the three new index constituents –included in node and edge feature vectors, respectively– are often influential in predicting the intraday spot volatility of all other assets during the period covered by our study. In particular, note that AMGN is included in the most influential subgraphs for other nodes with a frequency between approximately 40% and 80%, in both the validation and test sets. CRM and HON follow with frequencies that range approximately between 40% and 50% (for CRM) and 10% and 30% (for HON).

These findings may be explained by the fact that the training period of SpotV2Net, which ranges tentatively from mid-2020 to mid-2022, corresponds to eventful contingencies for these three companies. At that time, Amgen was conducting its research on the Covid-19 vaccine, Salesforce was experiencing an increase in the demand for cloud platforms, due to the rise in remote work caused

10 For the implementation, we use the Python package available at the link GNNExplainer. Details about the optimization algorithm are provided in Section 4 of Ying et al. (2019).

by the pandemic and the related lockdowns, and Honeywell was converting part of its manufacturing facilities to produce protective equipment for healthcare workers to reduce contagion risks. Furthermore, it is worth noting that the features of Verizon (VZ) and Walgreens (WBA) are also recognized as influential for nodes' predictions by GNNExplainer, with frequencies that range approximately between 10% and 30%. This may be explained by the fact that their businesses (telecommunications and retail pharmacy, respectively) were closely tied to pandemic-related contingencies. Overall, the GNNExplainer-based interpretation of the relationships learned by SpotV2Net during the training period may suggest the presence of a drag effect coming from the volatility of certain DJIA index components related to key economic sectors for the observed period.

7.4. Multi-step forecast

Accurate multi-step forecasting can be advantageous for high-frequency traders who wish not only to predict volatilities at the immediate next time step but also to extrapolate entire trajectories, enabling them to plan their operations with a broader, daily, temporal perspective. Accordingly, in this section, we extend the scope of application of the SpotV2Net model to obtain multiple-step-ahead forecasts. Specifically, we adapt the model's architecture to predict the entire daily volatility function on the 30-minute grid, which corresponds to 14 future observations for a 6.5-hour-long trading day.[11] Note that multi-step forecasts are all conditional on the information available at the end of the previous trading day (i.e., node and edge features of the graph at the end of the previous trading day). In other words, letting τ_0 denote the end of the previous day, the multi-step version of SpotV2Net does not use as input the predictions at times $\tau_1, \ldots, \tau_{h-1}$ to obtain the prediction at time τ_h, $1 < h \leq 14$. Instead, the model produces, at τ_0, a unique joint forecast of the values of the spot volatility function on the time grid $\tau_1, \ldots, \tau_{14}$, with a mesh size equal to 30 min. We refer to this type of multi-step forecast as a functional forecast. We evaluate the performance of SpotV2Net at producing such forecasts against the same alternative models considered in Section 7.2.[12]

To evaluate the accuracy of the functional forecasts, we use *global* loss functions. Specifically, we employ the average MSE of the multi-step prediction, that is,

$$\frac{1}{D_{\text{test}}} \sum_{j=1}^{D_{\text{test}}} \frac{1}{N} \sum_{i=1}^{N} \frac{1}{H} \sum_{h=1}^{H} \left(\widehat{V}_{i,j+h} - \widehat{V}^*_{i,j+h} \right)^2 \tag{7}$$

[11] In principle, it is possible to adapt SpotV2Net's architecture to produce multi-step forecasts at any point in time. For simplicity, we restrict our study to multi-step forecasts obtained at the end of a trading day for the entire next trading day.

[12] The ability to produce a functional forecast is characteristic of deep learning models, like SpotV2Net and the LSTM model. However, such an ability is not shared by the HAR-Spot model and the XGB model. Indeed, to obtain multi-step forecasts with these models, one has to resort to a recursive approach, where the one-step-ahead forecast for time τ_j is employed as an input to obtain the one-step-ahead forecast for time τ_{j+1}.

Table 5
Aggregate values of the average MSE and QLIKE in Eqs. (7) and (8) for the SpotV2Net model and the alternative models during the validation and test phases. An asterisk (*) indicates models that are included in the MCS at the 5% significance level.

	Validation		Test	
Multi-step model	MSE	QLIKE	MSE	QLIKE
HAR-Spot	4.453e−08	0.310	5.876e−08	0.407
XGB	4.156e−08	0.291	6.943e−08	0.478
LSTM	2.923e−08	0.204	5.467e−08	0.379
SpotV2Net-NE	2.643e−08	0.180	5.312e−08	0.368
SpotV2Net	2.145e−08*	0.142*	4.342e−08*	0.294*

Table 6
Comparison of multi-step forecasting accuracy using the Diebold–Mariano test with the average MSE (top) and QLIKE (bottom) losses on the validation set. Positive numbers indicate that the column model outperforms the row model. An asterisk (*) denotes a significance level of 5%.

Validation period (MSE)				
	XGB	LSTM	SpotV2Net-NE	SpotV2Net
HAR-Spot	7.12*	46.76*	48.21*	58.23*
XGB	–	28.55*	35.15*	47.36*
LSTM	–	–	5.64*	11.98*
SpotV2Net-NE	–	–	–	8.53*
Validation period (QLIKE)				
	XGB	LSTM	SpotV2Net-NE	SpotV2Net
HAR-Spot	5.41*	30.20*	37.05*	43.82*
XGB	–	24.79*	31.63*	38.37*
LSTM	–	–	6.83*	14.51*
SpotV2Net-NE	–	–	–	7.09*

and the average QLIKE, that is,

$$\frac{1}{D_{\text{test}}} \sum_{j=1}^{D_{\text{test}}} \frac{1}{N} \sum_{i=1}^{N} \frac{1}{H} \sum_{h=1}^{H} \left[\frac{\widehat{V}_{i,j+h}}{\widehat{V}^*_{i,j+h}} - \log \frac{\widehat{V}_{i,j+h}}{\widehat{V}^*_{i,j+h}} - 1 \right], \tag{8}$$

where $H = 14$ denotes the number of intraday 30-minute forecast steps, and D_{test} indicates the number of days in the test set. When the validation period is considered, D_{test} is replaced by D_{val}, which indicates the number of days in the validation period.

Table 5 summarizes the multi-step forecasting performance of the models employed in our study, in the validation and test phases. In the validation period, the ranking is the same as for single-step forecasts, in terms of aggregate values of both the average MSE and QLIKE. In the test period, however, the HAR-Spot model has a worse ranking than in the single-step exercise, as it outperforms only the XGB model. This finding is in line with the results by Christensen et al. (2023), suggesting that the relative forecasting accuracy of HAR-type models may deteriorate at longer forecasting horizons, compared to that of ML models. Table 5 also reports the results of the application of the MCS at the 95% confidence level. The outcome is analogous to the one obtained in the single-step case and supports the superior forecasting performance of SpotV2Net in the multi-step exercise as well.

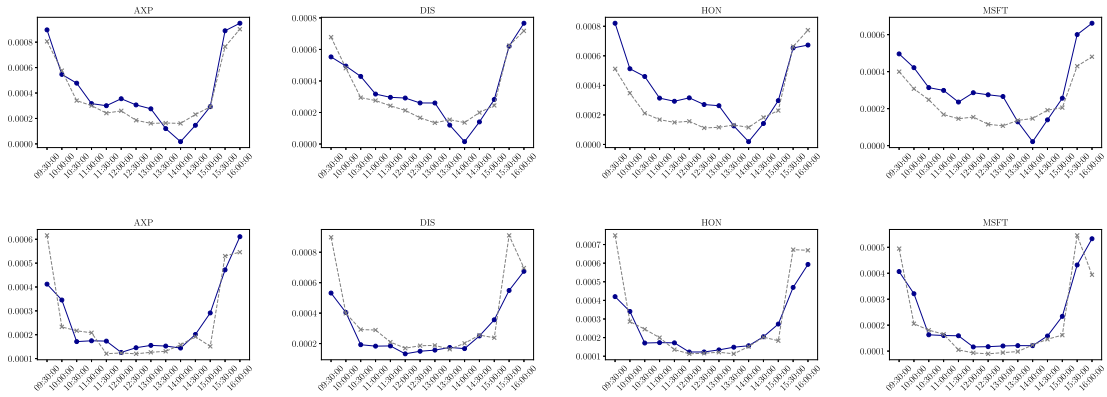

Fig. 5. Predicted (dotted blue) and actual (reconstructed, dashed grey) spot volatility path for AXP, DIS, HON, and MSFT. The first row refers to September 19, 2022, comprised in the validation period, and the second row to February 1, 2023, comprised in the test period. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Table 7
Comparison of multi-step forecasting accuracy using the Diebold–Mariano test with the average MSE (top) and QLIKE (bottom) losses on the test set. Positive numbers indicate that the column model outperforms the row model. An asterisk (*) denotes a significance level of 5%.

Test period (MSE)				
	XGB	LSTM	SpotV2Net-NE	SpotV2Net
HAR-Spot	−25.57*	9.80*	13.52*	33.16*
XGB	–	35.38*	39.10*	59.51*
LSTM	–	–	3.71*	24.61*
SpotV2Net-NE	–	–	–	21.01*
Test period (QLIKE)				
	XGB	LSTM	SpotV2Net-NE	SpotV2Net
HAR-Spot	−20.23*	7.90*	11.15*	28.57*
XGB	–	28.21*	31.34*	46.95*
LSTM	–	–	3.13*	21.34*
SpotV2Net-NE	–	–	–	17.15*

Moreover, the Diebold-Mariano test statistics are reported in Tables 6 and 7. The results suggest that the gain in forecasting accuracy provided by SpotV2Net is statistically significant at the 5% level for multi-step forecasts as well. Finally, to illustrate an example of the accuracy of the multi-step functional predictions produced by SpotV2Net, Fig. 5 shows a comparison of the latter with the actual reconstructed spot volatility path of four companies for two days, one in the validation set and one in the test set.

8. Conclusions

This paper contributes to research on volatility forecasting by introducing SpotV2Net, a model that employs a neural network with a GAT architecture to produce intraday forecasts of multivariate spot volatilities. Specifically, SpotV2Net leverages the interconnected nature of financial markets by modeling each asset as a node in a graph structure, where node and edge features represent, respectively, univariate and multivariate spot volatilities and volatilities of volatilities. Furthermore, the attention mechanism embedded in the GAT architecture allows SpotV2Net to optimally weigh the influence of specific subgraphs when producing spot volatility forecasts for a specific node.

The first novelty of this work is to address the rather unexplored topic of intraday volatility forecasting, which is relevant for its economic implications on intraday risk management, high-frequency trading, and early detection of market instability conditions. Specifically, it does so by focusing on the actual spot co-volatility matrix, estimated in a non-parametric setting by means of Fourier estimators, rather than by considering parametric volatility estimates or non-parametric integrated volatility estimates computed on short time horizons.

The second novelty of this work relates to the use of the spot co-volatilities of volatilities as edge features in the graph structure to model the dynamics of the dependence between two nodes and thereby capture spillover effects. Our findings related to single-step and multi-step forecasts of the spot volatility of the DIJA components suggest that including edge features in the attention mechanism that weighs the importance of subgraphs for predicting the future spot volatility of a specific node yields statistically significant gains in forecasting accuracy, compared to the use of a HAR-type model and other ML models.

The third novelty of our work is to address a typical problem related to ML techniques, that is, interpretability. In this regard, we used GNNExplainer, a model-agnostic tool designed for graph neural networks, to interpret the forecasts provided by the SpotV2Net model. As a result, we obtained insights into the subgraph structures influencing each node's volatility predictions and discovered the significant impact of the most recent additions to the DJIA index.

Note that this paper focused on predicting individual volatilities, rather than the entire co-volatility matrix, to avoid the curse of dimensionality and reduce computational costs. However, we remark that SpotV2Net's architecture is inherently flexible and thus could be adapted to forecast asset co-volatilities. We leave this extension,

which would be particularly beneficial for portfolio optimization and risk management purposes, to future research.

Another potentially interesting topic for future research is the development of recurrent networks with GAT layers. This represents an unexplored area of study, to the best of our knowledge. Indeed, integrating a recurrent neural network with the attention mechanism would allow for directly capturing the evolution of dynamic graphs over time, rather than relying on static graphs constructed using lagged features.

CRediT authorship contribution statement

Alessio Brini: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Giacomo Toscano:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Alessio Brini reports financial support was provided by Institut Louis Bachelier. Giacomo Toscano reports financial support was provided by Institut Louis Bachelier.

Acknowledgments

The authors would like to thank the Associate Editor and two anonymous referees for their valuable comments and suggestions. Furthermore, the authors gratefully acknowledge financial support from the Institut Louis Bachelier 2022 grant 'Risk management in times of unprecedented geo-political volatility: a machine learning approach'.

Appendix A. Alternative models

A.1. Heterogeneous autoregressive (HAR) spot model

We adapt the HAR model by Corsi (2009) to forecast intraday spot volatilities. The resulting model, which we call HAR-Spot, assumes a linear relationship between the 30-minute-ahead spot volatility and three regressors: the current spot volatility, the average spot volatility between lag 1 (30 min) and lag 7 (3.5 h), and the average spot volatility between lag 8 (4 h) and lag 13 (6.5 h, that is, one trading day). This way, in the spirit of the original model by Corsi (2009), the HAR-Spot model captures the persistence of the spot volatility in a parsimonious way. Specifically, we implement the following panel version of the HAR-Spot model. For the ith asset, the HAR-Spot model reads as

$$\begin{aligned}\widehat{V}_{i,b+1} = \mu + \phi_1 \widehat{V}_{i,b} + \phi_2 \left(\frac{1}{7}\sum_{l=1}^{7} \widehat{V}_{i,b-l}\right) \\ + \phi_3 \left(\frac{1}{7}\sum_{l=8}^{13} \widehat{V}_{i,b-l}\right) \\ + \sum_{k\neq i}\left[\theta_1 \widehat{V}_{k,t} + \theta_2 \left(\frac{1}{7}\sum_{l=1}^{7} \widehat{V}_{k,b-l}\right)\right. \\ \left. + \theta_3 \left(\frac{1}{7}\sum_{l=8}^{13} \widehat{V}_{k,b-l}\right)\right] + \epsilon_{i,b},\end{aligned}$$

where $\sum_{k\neq i}[\cdots]$ denotes the contribution from the lagged volatilities of all other assets $k \neq i$.

A.2. Extreme gradient boosting (XGB)

Ensembles of weak learners, such as regression trees (Breiman, Friedman, Olshen, & Stone, 1984), can effectively handle non-linear interactions between predictors. Specifically, XGB (Chen & Guestrin, 2016) is a tree-based ensemble algorithm that has proven effective at addressing prediction problems based on tabular data (Shwartz-Ziv & Armon, 2022). Hence, our approach is to input the regressors to such an algorithm in the same form we input them to fit the HAR-Spot model. Letting $u_{i,b}$ denote the vector of input features for the ith asset at time τ_b, the XGB model is specified as

$$F_i(u_{i,b}) = \sum_{j=1}^{J} f_j(u_{i,b}), \quad f_j \in \mathcal{R},$$

where J is the number of estimators composing the ensemble, and $\mathcal{R}$ is the space of regression trees. The tree ensemble model is trained sequentially. The boosting technique (Friedman, 2001) indicates that trees are added to minimize the errors made by previously fitted trees until no further improvements are achieved. The optimization procedure builds trees as a forward mechanism, where every step reduces the error of the previous iteration. We initialize the ensemble with a single regression tree and then iteratively add new trees that minimize the error made by the previous tree by gradient descent.

We chose to train XGB using the same inputs as for the HAR-Spot model because training the latter using all the inputs considered in the node and edge features of SpotV2Net resulted in lower performance. This choice aligns with findings in the literature suggesting that when the feature space is too large, gradient boosting techniques, including XGB, are more prone to overfitting (Natekin & Knoll, 2013). Additionally, limiting the number of features to just the volatilities, which represent the objects of forecasting, simplifies the multi-step forecasting process. Like HAR-Spot, XGB performs multi-step forecasts recursively. Hence, including additional features in the input space, such as, e.g., the co-volatilities, would require XGB to recursively forecast not only volatilities but also the additional features.

A.3. Long short-term memory (LSTM) neural network

Recurrent neural networks (RNNs), see Hochreiter and Schmidhuber (1997), are models designed to retain information about sequential data, making them particularly effective in the financial context where one usually forecasts future values based on historical data. A performing version of an RNN is the LSTM network (Hochreiter & Schmidhuber, 1997), which has contributed to significant breakthroughs across several fields, including natural language processing, time series forecasting, and generative modeling (Gers, Eck, & Schmidhuber, 2001; Siami-Namini, Tavakoli, & Namin, 2019).

The LSTM architecture proposes a memory block to capture long-term dependencies more effectively. Within each LSTM unit, a standard transformation of the original input occurs. Consider the vector of input features $\mathbf{x}_{\tau_b}$ at time τ_b which comprises all companies' univariate volatilities and co-volatilities. The LSTM model accepts these vectors $\mathbf{x}_{\tau_b}$ up to a given lag L. Note that we input only spot volatilities and co-volatilities time series in the LSTM algorithm because our experiments suggested that also inputting univariate and multivariate co-volatilities of volatilities and their lags worsens the model's performance. We attribute this performance decrease to the large dimensionality of the input size. In contrast, SpotV2Net uses this information efficiently by distributing it into a graph structure with node and edge features. LSTM has no knowledge of such a structure and thus infers relationships by combining information through different layers, which proved to be limited in our empirical experiment.

The transformation performed by the LSTM on the vectors $\mathbf{x}_{\tau_b}$ at time τ_b reads as

$$
\begin{aligned}
\mathbf{f}_{\tau_b} &= \sigma_g\left(\mathbf{W}_f\mathbf{x}_{\tau_b} + U_f\mathbf{h}_{\tau_{b-1}} + \mathbf{b}_f\right)\\
\mathbf{i}_{\tau_b} &= \sigma_g\left(\mathbf{W}_i\mathbf{x}_{\tau_b} + U_i\mathbf{h}_{\tau_{b-1}} + \mathbf{b}_i\right)\\
\mathbf{o}_{\tau_b} &= \sigma_g\left(\mathbf{W}_o\mathbf{x}_{\tau_b} + U_o\mathbf{h}_{\tau_{b-1}} + \mathbf{b}_o\right)\\
\tilde{\mathbf{c}}_t &= \sigma_c\left(\mathbf{W}_c\mathbf{x}_t + U_c\mathbf{h}_{\tau_{b-1}} + \mathbf{b}_c\right)\\
\mathbf{c}_{\tau_b} &= \mathbf{f}_{\tau_b} \circ \mathbf{c}_{\tau_{b-1}} + \mathbf{i}_{\tau_b} \circ \tilde{\mathbf{c}}_{\tau_b}\\
\mathbf{h}_{\tau_b} &= \mathbf{o}_{\tau_b} \circ \sigma_h\left(\mathbf{c}_{\tau_b}\right)
\end{aligned}
$$

where $\mathbf{x}_{\tau_b}$ is the input vector, $\mathbf{f}_{\tau_b}$ is the forget gate's activation vector, $\mathbf{i}_{\tau_b}$ is the update gate's activation vector, $\mathbf{o}_{\tau_b}$ is the output gate's activation vector, $\tilde{\mathbf{c}}_{\tau_b}$ is the cell input activation vector, $\mathbf{c}_{\tau_b}$ is the cell state vector, and $\mathbf{h}_{\tau_b}$ is the hidden state vector, i.e., the output vector of the LSTM unit. Further, $\circ$ is the Hadamard product function, σ_g is the sigmoid function, and σ_c and σ_h are hyperbolic tangent functions. Finally, $\mathbf{W}_{(\cdot)}$ and $\mathbf{b}_{(\cdot)}$ refer to weight matrices and bias vectors that need to be estimated by training the model, while $\mathbf{U}_{(\cdot)}$ refers to the weight matrices that are applied to the hidden state vector from the previous time step $\mathbf{h}_{\tau_{b-1}}$.

Appendix B. Hyperparameters

For the SpotV2Net, XGB and LSTM models, we utilized the Optuna framework implemented in Python, which performs a search over a predefined grid of hyperparameters. This technique is commonly employed for optimizing neural network-based models. In our hyperparameter optimization process with Optuna, we employ the tree-structured Parzen estimator (TPE) algorithm as our sampler (Bergstra, Bardenet, Bengio, & Kégl, 2011; Bergstra & Bengio, 2012). The TPE algorithm enhances the efficiency of the search by using a Bayesian optimization technique. During each trial, for each parameter, TPE fits one Gaussian mixture model (GMM) (Dempster, Laird, & Rubin, 1977) $l(x)$ to the set of parameter values associated with the best objective values, and another GMM $g(x)$ to the remaining parameter values. It then chooses the parameter value x that maximizes the ratio $\frac{l(x)}{g(x)}$. By choosing the hyperparameter values that maximize this ratio, the TPE focuses the search on regions where the likelihood of improving the objective function is higher. This approach ensures that our hyperparameter tuning is both effective and computationally efficient.

The grid for Optuna is constructed by specifying a meaningful set of possible values for each hyperparameter, which we report together with the optimized values in Tables 8, 10, and 9 for the SpotV2Net, LSTM, and XGB models, respectively. Due to memory constraints on the NVIDIA GeForce RTX 2080 Ti with 12 GB of memory that we used to train the deep learning models, we were

Table 8
Hyperparameter grid search options and optimized values for the single-step and multi-step SpotV2Net models.

Hyperparameter	Grid search options	Single-step opt. val.	Multi-step opt. val.
Loss Function	–	MSE	MSE
Activation Function (σ)	ReLu, Tanh, Sigmoid	ReLu	ReLu
Batch Size	32, 64, 128, 256	128	128
Concatenate Heads	True, False	True	True
Number Hidden Layers	1, 2, 3	2	2
Dimension Hidden Layers (M')	from 50 to 600 (step 50)	[400, 200]	[400, 400]
Dropout (Architecture)	0.0 to 0.7 (step 0.1)	0.1	0.2
Dropout (Attention)	0.0 to 0.7 (step 0.1)	0.1	0.0
Learning Rate	$\{1, 2.5, 5\} \times 10^{-2 \text{ to } -5}$	1×10^{-4}	5×10^{-5}
Negative Slope LeakyReLu (c)	0.05, 0.1, 0.2, 0.5, 0.6, 0.8	0.1	0.1
Epochs	60 to 200 (step 20)	120	120
Number of Heads (K)	2, 3, 4, 5, 6, 7	4	5
Optimizer	RMSProp, Adam, AdamW	AdamW	AdamW
Output Dim	–	1	14
Number of Lags (L)	14 to 112 (step 14)	42	42

Table 9
Hyperparameter grid search options and optimized values for the XGB model.

Hyperparameter	Grid search options	Opt. val.
Subsample	0.6, 0.7, 0.8, 0.9, 1	0.7
Regularization λ	0.5, 1, 1.5, 2	1.5
Regularization α	0, 0.1, 0.5, 1	0
Number of Estimators (J)	50, 100, 150, 200, 300, 400, 500, 600	400
Minimum Child Weight	1, 3, 5, 7	5
Max Depth	3, 4, 5, 6, 7, 8	5
Learning Rate	0.001, 0.01, 0.05, 0.1, 0.2	0.2
Gamma	0, 0.05, 0.1, 0.2, 0.3, 0.4	0
Colsample by Tree	0.5, 0.6, 0.7, 0.8, 0.9, 1	1

Table 10
Hyperparameter grid search options and optimized values for the single-step and multi-step LSTM models.

Hyperparameter	Grid search options	Single-step opt. val.	Multi-step opt. val.
Loss Function	–	MSE	MSE
Batch Size	32, 64, 128, 256	64	64
Dropout (Architecture)	0.0 to 0.7 (step 0.1)	0.4	0.3
Number Hidden Layers	1, 2, 3	2	2
Dimension Hidden Layers	from 50 to 600 (step 50)	[400, 200]	[400, 400]
Learning Rate	$\{1, 2.5, 5\} \times 10^{-2 \text{ to } -5}$	5×10^{-4}	5×10^{-4}
Optimizer	RMSProp, Adam, AdamW	AdamW	AdamW
Output Dim	–	1	14

unable to expand the grid search to include configurations with more than three layers, larger layer sizes, a higher number of heads, or larger batch updates, as these modifications would exceed the available computational resources. The optimal values reported are the result of this search process, indicating the configuration that achieved the best performance on the validation set, as shown in Table 1. The hyperparameters are optimized for all models by evaluating their performance on a validation set, ensuring that the test set remains unseen during the training and tuning phases. This strategy prevents any bias towards the test set and ensures that our evaluation metrics reflect the models' ability to generalize to new data.

Data and code availability

The data used in this study is proprietary and comes from the Trade and Quote (TAQ) database via Wharton Research Data Services (WRDS). The data must be purchased separately through WRDS, and redistribution is not permitted. The source code used for this study is publicly available at: https://github.com/Alessiobrini/SpotV2Net.

References

Allaj, E., & Sanfelici, S. (2023). Early warning systems for identifying financial instability. *International Journal of Forecasting*, *39*(4), 1777–1803.

Andersen, T. G., Bollerslev, T., Christoffersen, P. F., & Diebold, F. X. (2006). Volatility and correlation forecasting. *Handbook of Economic Forecasting*, *1*, 777–878.

Andersen, T. G., Bollerslev, T., Diebold, F. X., & Ebens, H. (2001). The distribution of realized stock return volatility. *Journal of Financial Economics*, *61*(1), 43–76.

Bandi, F. M., Russell, J. R., & Yang, C. (2008). Realized volatility forecasting and option pricing. *Journal of Econometrics*, *147*(1), 34–46.

Barucci, E., & Mancino, M. E. (2010). Computation of volatility in stochastic volatility models with high-frequency data. *International Journal of Theoretical and Applied Finance*, *13*(05), 767–787.

Bauwens, L., Laurent, S., & Rombouts, J. V. (2006). Multivariate GARCH models: A survey. *Journal of Applied Econometrics*, *21*(1), 79–109.

Becker, R., Clements, A. E., Doolan, M. B., & Hurn, A. S. (2015). Selecting volatility forecasting models for portfolio allocation purposes. *International Journal of Forecasting*, *31*(3), 849–861.

Behrendt, S., & Schmidt, A. (2018). The Twitter myth revisited: Intraday investor sentiment Twitter activity and individual-level stock return volatility. *Journal of Banking & Finance*, *96*, 355–367.

Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for hyper-parameter optimization. *Advances in Neural Information Processing Systems*, *24*.

Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, *13*(2).

Bollerslev, T., Engle, R. F., & Wooldridge, J. M. (1988). A capital asset pricing model with time-varying covariances. *Journal of Political Economy*, *96*(1), 116–131.

Bollerslev, T., Meddahi, N., & Nyawa, S. (2019). High-dimensional multivariate realized volatility estimation. *Journal of Econometrics*, *212*(1), 116–136.

Bollerslev, T., Tauchen, G., & Zhou, H. (2009). Expected stock returns and variance risk premia. *The Review of Financial Studies*, *22*(11), 4463–4492.

Brailsford, T. J., & Faff, R. W. (1996). An evaluation of volatility forecasting techniques. *Journal of Banking & Finance*, *20*(3), 419–438.

Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). Cart. *Classification and Regression Trees*.

Bucci, A. (2020). Realized volatility forecasting with neural networks. *Journal of Financial Econometrics*, *18*(3), 502–531.

Caldeira, J. F., Moura, G. V., Nogales, F. J., & Santos, A. A. (2017). Combining multivariate volatility forecasts: An economic-based approach. *Journal of Financial Econometrics*, *15*(2), 247–285.

Callot, L. A., Kock, A. B., & Medeiros, M. C. (2017). Modeling and forecasting large realized covariance matrices and portfolio choice. *Journal of Applied Econometrics*, *32*(1), 140–158.

Campisi, G., Muzzioli, S., & Baets, B. D. (2023). A comparison of machine learning methods for predicting the direction of the US stock market on the basis of volatility indices. *International Journal of Forecasting*.

Catania, L., & Proietti, T. (2020). Forecasting volatility with time-varying leverage and volatility of volatility effects. *International Journal of Forecasting*, *36*(4), 1301–1317.

Chen, T.-F., Chordia, T., Chung, S.-L., & Lin, J.-C. (2021). Volatility-of-volatility risk in asset pricing. *The Review of Asset Pricing Studies*, *12*(1), 289–335.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 785–794).

Chen, L., Pelger, M., & Zhu, J. (2023). Deep learning in asset pricing. *Management Science*.

Chen, Q., & Robert, C.-Y. (2022). Multivariate realized volatility forecasting with graph neural network. In *Proceedings of the third ACM international conference on AI in finance* (pp. 156–164).

Cheng, R., & Li, Q. (2021). Modeling the momentum spillover effect for stock prediction via attribute-driven graph attention networks. *Vol. 35*, In *Proceedings of the AAAI conference on artificial intelligence* (pp. 55–62).

Cheng, D., Yang, F., Xiang, S., & Liu, J. (2022). Financial time series forecasting with multi-modality graph neural network. *Pattern Recognition*, *121*, Article 108218.

Christensen, K., Siggaard, M., & Veliyev, B. (2023). A machine learning approach to volatility forecasting. *Journal of Financial Econometrics*, *21*(5), 1680–1727.

Christoffersen, P. F., & Diebold, F. X. (2000). How relevant is volatility forecasting for financial risk management?. *The Review of Economics and Statistics*, *82*(1), 12–22.

Corsi, F. (2009). A simple approximate long-memory model of realized volatility. *Journal of Financial Econometrics*, *7*(2), 174–196.

Corsi, F., Mittnik, S., Pigorsch, C., & Pigorsch, U. (2008). The volatility of realized volatility. *Econometric Reviews*, *27*(1–3), 46–78.

Defferrard, M., Bresson, X., & Vandergheynst, P. (2016). Convolutional neural networks on graphs with fast localized spectral filtering. *Advances in Neural Information Processing Systems*, *29*.

Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society. Series B. Statistical Methodology*, *39*(1), 1–22.

Diao, Z., Wang, X., Zhang, D., Liu, Y., Xie, K., & He, S. (2019). Dynamic spatial-temporal graph convolutional neural networks for traffic forecasting. *Vol. 33*, In *Proceedings of the AAAI conference on artificial intelligence* (pp. 890–897).

Diebold, F. X., & Mariano, R. S. (2002). Comparing predictive accuracy. *Journal of Business & Economic Statistics*, *20*(1), 134–144.

Ding, Y. D. (2023). A simple joint model for returns, volatility and volatility of volatility.. *Journal of Econometrics*, *232*(2), 521–543.

Djanga, E., Cucuringu, M., & Zhang, C. (2023). Cryptocurrency volatility forecasting using commonality in intraday volatility. In *Proceedings of the Fourth ACM International Conference on AI in Finance* (pp. 436–444).

Engle, R. F., & Kroner, K. F. (1995). Multivariate simultaneous generalized ARCH. *Econometric Theory*, *11*(1), 122–150.

Engle, R. F., & Sokalska, M. E. (2012). Forecasting intraday volatility in the US equity market, multiplicative component GARCH. *Journal of Financial Econometrics*, *10*(1), 54–83.

Fan, W., Ma, Y., Li, Q., He, Y., Zhao, E., Tang, J., et al. (2019). Graph neural networks for social recommendation. In *The world wide web conference* (pp. 417–426).

Fassas, A. P., & Siriopoulos, C. (2019). Intraday price discovery and volatility spillovers in an emerging market. *International Review of Economics and Finance*, *59*, 333–346.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, *29*(5), 1189–1232.

Gers, F. A., Eck, D., & Schmidhuber, J. (2001). Applying LSTM to time series predictable through time-window approaches. In *International conference on artificial neural networks* (pp. 669–676). Springer.

Goldstein, M., Kwan, A., & Philip, R. (2023). High-frequency trading strategies. *Management Science*, *69*, 4413–4434.

Golosnoy, V., Gribisch, B., & Liesenfeld, R. (2015). Intra-daily volatility spillovers in international stock markets. *Journal of International Money and Finance*, *53*, 95–114.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Gopal, A., & Chang, C. (2021). Discovering supply chain links with augmented intelligence. arXiv preprint arXiv:2111.01878.

Gouriéroux, C., Jasiak, J., & Sufana, R. (2009). The Wishart autoregressive process of multivariate stochastic volatility. *Journal of Econometrics*, *150*(2), 167–181.

Granger, C. W., & Joyeux, R. (1980). An introduction to long-memory time series models and fractional differencing. *Journal of Time Series Analysis*, *1*(1), 15–29.

Gu, S., Kelly, B., & Xiu, D. (2020). Empirical asset pricing via machine learning. *The Review of Financial Studies*, *33*(5), 2223–2273.

Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems*, *30*.

Hansen, P. R., Lunde, A., & Nason, J. M. (2011). The model confidence set. *Econometrica*, *79*(2), 453–497.

Heaton, J. B., Polson, N. G., & Witte, J. H. (2017). Deep learning for finance: Deep portfolios. *Applied Stochastic Models in Business and Industry*, *33*(1), 3–12.

Herskovic, B., Kelly, B., Lustig, H., & Va. Nieuwerburgh, S. (2016). The common factor in idiosyncratic volatility: Quantitative asset pricing implications. *Journal of Financial Economics*, *119*(2), 249–283.

Herskovic, B., Kelly, B., Lustig, H., & Va. Nieuwerburgh, S. (2020). Firm volatility in granular networks. *Journal of Political Economy*, *128*(11), 4097–4162.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780.

Huang, T., Dong, Y., Ding, M., Yang, Z., Feng, W., Wang, X., et al. (2021). MixGCF: An improved training method for graph neural network-based recommender systems. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining* (pp. 665–674).

Huang, D., Schlag, C., Shaliastovich, I., & Thimme, J. (2019). Volatility-of-volatility risk. *Journal of Financial and Quantitative Analysis*, *54*(6), 2423–2452.

Jaegle, A., Gimeno, F., Brock, A., Vinyals, O., Zisserman, A., & Carreira, J. (2021). Perceiver: General perception with iterative attention. In *International conference on machine learning* (pp. 4651–4664). PMLR.

Jawadi, F., Louhichi, W., & Idi. Cheffou, A. (2015). Intraday bidirectional volatility spillover across international stock markets: Does the global financial crisis matter?. *Applied Economics*, *47*, 3633–3650.

Jiang, W., & Luo, J. (2022). Graph neural network for traffic forecasting: A survey. *Expert Systems with Applications*, *207*, Article 117921.

Katsiampa, P., Corbet, S., & Lucey, B. (2019). High-frequency volatility co-movements in cryptocurrency markets. *Journal of International Financial Markets, Institutions and Money*, *62*, 35–52.

Kim, R., So, C. H., Jeong, M., Lee, S., Kim, J., & Kang, J. (2019). HATS: A hierarchical graph attention network for stock movement prediction. arXiv preprint arXiv:1908.07999.

Kipf, T. N., & Welling, M. (2016). Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907.

LeCun, Y., Bengio, Y., et al. (1995). Convolutional networks for images, speech, and time series. *The Handbook of Brain Theory and Neural Networks*, *3361*(10), 1995.

Li, L. (2022). The dynamic interrelations of oil-equity implied volatility indexes under low and high volatility-of-volatility risk. *Energy Economics*, *105*, Article 105756.

Li, M., & Zhu, Z. (2021). Spatial-temporal fusion graph neural networks for traffic flow forecasting. *Vol. 35*, In *Proceedings of the AAAI conference on artificial intelligence* (pp. 4189–4196).

Ligot, S., Gillet, R., & Veryzhenko, I. (2021). Intraday volatility smile: Effects of fragmentation and high-frequency trading on price efficiency. *Journal of International Financial Markets, Institutions and Money*, *75*, Article 101437.

Lin, W., & Taamouti, A. (2024). Portfolio selection under non-Gaussianity and systemic risk: A machine learning based forecasting approach. *International Journal of Forecasting*, *40*(3), 1179–1188.

Ling, S., & McAleer, M. (2003). Asymptotic theory for a vector ARMA-GARCH model. *Econometric Theory*, *19*(2), 280–310.

Liu, Y. (2019). Novel volatility forecasting using deep learning–long short term memory recurrent neural networks. *Expert Systems with Applications*, *132*, 99–109.

Liu, J., Cheng, D., & Jiang, C. (2023). Preventing attacks in interbank credit rating with selective-aware graph neural network. In *Proceedings of the thirty-second international joint conference on artificial intelligence* (pp. 6085–6093).

Liu, F., Pantelous, A. A., & von Mettenheim, H.-J. (2018). Forecasting and trading high-frequency volatility on large indices. *Quantitative Finance*, *18*, 737–748.

Ma, Y., Han, R., & Wang, W. (2021). Portfolio optimization with return prediction using deep learning and machine learning. *Expert Systems with Applications*, *165*, Article 113973.

Maas, A. L., Hannun, A. Y., Ng, A. Y., et al. (2013). Rectifier nonlinearities improve neural network acoustic models. *Vol. 30*, In *Proceedings of the international conference on machine learning* (p. 3). Atlanta, GA.

Madhusudan, K., & Samit, P. (2019). Intraday portfolio risk management using VaR and CVaR: A CGARCH-EVT-Copula approach. *International Journal of Forecasting*, *35*(2), 699–709.

Malliavin, P., & Mancino, M. E. (2002). Fourier series method for measurement of multivariate volatilities. *Finance and Stochastics*, *6*(1), 49–61.

Malliavin, P., & Mancino, M. E. (2009). A Fourier transform method for nonparametric estimation of multivariate volatility. *The Annals of Statistics*, *37*(4), 1983–2010.

Mancino, M. E., Mariotti, T., & Toscano, G. (2022). Asymptotic normality for the Fourier spot volatility estimator in the presence of microstructure noise. arXiv preprint arXiv:2209.08967.

Mancino, M. E., & Recchioni, M. C. (2015). Fourier spot volatility estimator: Asymptotic normality and efficiency with liquid and illiquid high-frequency data. *PLoS One*, *10*(9), Article e0139041.

Mariotti, T., Lillo, F., & Toscano, G. (2023). From zero-intelligence to queue-reactive: Limit-order-book modeling for high-frequency volatility estimation and optimal execution. *Quantitative Finance*, *23*(3), 367–388.

Menghani, G. (2023). Efficient deep learning: A survey on making deep learning models smaller, faster, and better. *ACM Computing Surveys*, *55*(12), 1–37.

Min, S., Gao, Z., Peng, J., Wang, L., Qin, K., & Fang, B. (2021). STGSN – a spatial-temporal graph neural network framework for time-evolving social networks. *Knowledge-Based Systems*, *214*, Article 106746.

Monken, A., Haberkorn, F., Gopinath, M., Freeman, L., & Batarseh, F. A. (2021). Graph neural networks for modeling causality in international trade. *Vol. 34*, In *The international FLAIRS conference proceedings*.

Naeem, M. A., Karim, S., Yarovaya, L., & Lucey, B. M. (2023). COVID-induced sentiment and the intraday volatility spillovers between energy and other ETFs. *Energy Economics*, *122*, Article 106677.

Natekin, A., & Knoll, A. (2013). Gradient boosting machines, a tutorial. *Frontiers in Neurorobotics*, *7*(21).

Nishimura, Y., & Sun, B. (2018). The intraday volatility spillover index approach and an application in the Brexit vote. *Journal of International Financial Markets, Institutions and Money*, *55*, 241–253.

Panford-Quainoo, K., Bose, A. J., & Defferrard, M. (2020). Bilateral trade modelling with graph neural networks. In *ICLR workshop on practical ML for developing countries*.

Poon, S.-H., & Granger, C. W. J. (2003). Forecasting volatility in financial markets: A review. *Journal of Economic Literature*, *41*(2), 478–539.

Reisenhofer, R., Bayer, X., & Hautsch, N. (2022). HARNet: A convolutional neural network for realized volatility forecasting. arXiv preprint arXiv:2205.07719.

Rice, G., Wirjanto, T., & Zhao, Y. (2020). Forecasting value at risk with intra-day return curves. *International Journal of Forecasting*, *36*, 1023–1038.

Rossi, E., & Fantazzini, D. (2015). Long memory and periodicity in intraday volatility. *Journal of Financial Econometrics*, *13*(4), 922–961.

Rumelhart, D. E., Hinton, G. E., Williams, R. J., et al. (1985). Learning internal representations by error propagation.

Sanfelici, S., & Toscano, G. (2024). The Fourier-Malliavin volatility (FMVol) MATLAB library. *Mathematics and Computers in Simulation*, *226*, 338–353.

Satchell, S., & Knight, J. (2011). *Forecasting volatility in the financial markets*. Elsevier.

Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., & Monfardini, G. (2008). The graph neural network model. *IEEE Transactions on Neural Networks*, *20*(1), 61–80.

Shi, B., Dong, B., Xu, Y., Wang, J., Wang, Y., & Zheng, Q. (2023). An edge feature aware heterogeneous graph neural network model to support tax evasion detection. *Expert Systems with Applications*, *213*, Article 118903.

Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, *81*, 84–90.

Siami-Namini, S., Tavakoli, N., & Namin, A. S. (2019). The performance of LSTM and BiLSTM in forecasting time series. In *2019 IEEE international conference on big data* (pp. 3285–3292). IEEE.

Taylor, S. J., & Xu, X. (1997). The incremental volatility information in one million foreign exchange quotations. *Journal of Empirical Finance*, *4*(4), 317–340.

Toscano, G. (2022). The price-leverage covariation as a measure of the response of the leverage effect to price and volatility changes. *Applied Stochastic Models in Business and Industry*, *38*(3), 497–511.

Toscano, G., Livieri, G., Mancino, M. E., & Marmi, S. (2022). Volatility of volatility estimation: Central limit theorems for the Fourier transform estimator and empirical study of the daily time series stylized facts. *Journal of Financial Econometrics*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2017). Graph attention networks. arXiv preprint arXiv:1710.10903.

Wang, J., Zhang, S., Xiao, Y., & Song, R. (2021). A review on graph neural network methods in financial applications. arXiv preprint arXiv:2111.15367.

Wilms, I., Rombouts, J., & Croux, C. (2021). Multivariate volatility forecasts for stock market indices. *International Journal of Forecasting*, *37*(2), 484–499.

Wu, Q., Brinton, C. G., Zhang, Z., Pizzoferrato, A., Liu, Z., & Cucuringu, M. (2021). Equity2Vec: End-to-end deep learning framework for cross-sectional asset pricing. In *Proceedings of the second ACM international conference on AI in finance* (44), (pp. 1–9).

Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Philip, S. Y. (2020a). A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, *32*(1), 4–24.

Wu, Z., Pan, S., Long, G., Jiang, J., Chang, X., & Zhang, C. (2020b). Connecting the dots: Multivariate time series forecasting with graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 753–763).

Xia, L., Huang, C., Xu, Y., Dai, P., & Bo, L. (2022). Multi-behavior graph neural networks for recommender system. *IEEE Transactions on Neural Networks and Learning Systems*, *35*(4), 5473–5487.

Xiong, R., Nichols, E. P., & Shen, Y. (2015). Deep learning stock volatility with google domestic trends. arXiv preprint arXiv:1512.04916.

Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2018). How powerful are graph neural networks?. arXiv preprint arXiv:1810.00826.

Xue, Y., & Gençay, R. (2012). Trading frequency and volatility clustering. *Journal of Banking & Finance*, *36*, 760–773.

Ying, Z., Bourgeois, D., You, J., Zitnik, M., & Leskovec, J. (2019). GNNExplainer: Generating explanations for graph neural networks. *Advances in Neural Information Processing Systems*, *32*.

Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W. L., & Leskovec, J. (2018). Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 974–983).

Ying, X., Xu, C., Gao, J., Wang, J., & Li, Z. (2020). Time-aware graph relational attention network for stock recommendation. In *Proceedings of the 29th ACM international conference on information & knowledge management* (pp. 2281–2284).

Zhang, W., Chen, Z., Miao, J., & Liu, X. (2022b). Research on graph neural network in stock market. *Procedia Computer Science*, *214*, 786–792.

Zhang, C., Pu, X. S., Cucuringu, M., & Dong, X. (2022a). Graph-based methods for forecasting realized covariances. Available at SSRN.

Zhang, C., Pu, X. S., Cucuringu, M., & Dong, X. (2023a). Graph neural networks for forecasting realized volatility with nonlinear spillover effects. Available at SSRN.

Zhang, C., Zhang, Y., Cucuringu, M., & Qian, Z. (2023b). Volatility forecasting with machine learning and intraday commonality. *Journal of Financial Econometrics*, *22*(2), 492–530.

Zhang, Z., Zohren, S., & Roberts, S. (2020). Deep learning for portfolio optimization. *The Journal of Financial Data Science*.

Zhao, Y., Wei, S., Guo, Y., Yang, Q., Chen, X., Li, Q., et al. (2022). Combining intra-risk and contagion risk for enterprise bankruptcy prediction using graph neural networks. arXiv preprint arXiv:2202.03874.

Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., et al. (2020). Graph neural networks: A review of methods and applications. *AI Open*, *1*, 57–81.

Zhu, H., Bai, L., He, L., & Liu, Z. (2023). Forecasting realized volatility with machine learning: Panel data perspective. *Journal of Empirical Finance*, *73*, 251–271.